

Every question, solved and explained

## How this is arranged

All **204 questions** from the lecture slides, the reference sheets and both real tests. Grouped by topic, and within each topic ordered **easy first**, so the definitions come before the two-step calculations that assume them.

Every question carries the source it came from. Where the same question was set twice, it appears once and says where else it was asked. The answer is marked, and every other option has a line saying why it fails.

## 1 Introduction to Python Programming

1.1

EASY

Manual O.1 Q2

A Python script contains a syntax error on its last line. What happens when you run it?

- a. The first lines run, then the error is reported when that line is reached
- b. Nothing runs: the whole file is checked before any of it executes
- c. The error is ignored and the script finishes normally
- d. The line is skipped and the rest of the script continues

✗ The first lines run, then the error is reported when that line is reached

Wrong, and it is the trap. This is true of a NAME error or a type error, which are found only when the line runs, but a SYNTAX error is found while the file is being parsed, before anything executes.

✓ Nothing runs: the whole file is checked before any of it executes

Correct. Python parses the whole file first. A syntax error anywhere stops it before the first line runs, which is why a stray colon can produce no output at all.

✗ The error is ignored and the script finishes normally

Wrong. Python never ignores an error. Something is always reported.

✗ The line is skipped and the rest of the script continues

Wrong. Python does not skip statements it cannot parse. There is no "carry on regardless" mode.

The answer is Nothing runs: the whole file is checked before any of it executes. Syntax errors are found at parse time, before execution. Runtime errors are found when their line runs. Knowing which is which explains why some faulty scripts print nothing at all.

What is the correct way to run a saved file called `grades.py` from the command line?

- a. `python grades.py`
- b. `run grades.py`
- c. `grades.py run`
- d. `execute python grades.py`

✓ `python grades.py`

Correct. You launch the interpreter, `python`, and give it the file to run as its argument.

✗ `run grades.py`

Wrong. There is no `run` command. The interpreter is the program you launch, and the script is what you hand it.

✗ `grades.py run`

Wrong. The order is wrong and `run` is not a Python word.

✗ `execute python grades.py`

Wrong. There is no `execute` command wrapping it.

The answer is `python grades.py`. `python <filename>`. The interpreter is the program; your file is the input to it.

**Which line is a valid single-line comment in Python?**

- a. `// this is a comment`
- b. `# this is a comment`
- c. `/* this is a comment */`
- d. `-- this is a comment`

**X** `// this is a comment`

Wrong. Two slashes are the C, Java and JavaScript comment. In Python two slashes are the floor division operator, so this line is an error.

**✓** `# this is a comment`

Correct. The hash starts a comment that runs to the end of the line.

**X** `/* this is a comment */`

Wrong. Slash-star is the C block comment. Python has no block comment syntax at all.

**X** `-- this is a comment`

Wrong. Two hyphens are the SQL comment, which matters here because this course also writes SQL through Python. The two languages do not share it.

The answer is `# this is a comment`. Hash to end of line. Python has no block comment: a run of hashes, or a string used as a docstring, is how several lines are done.

### In Python, what does indentation do?

- a. Nothing: it is style only, as in C or Java
- b. It sets how far output is printed from the left edge
- c. It marks which statements belong to a block, and is part of the syntax
- d. It is required only inside functions

**✗** Nothing: it is style only, as in C or Java

Wrong, and it is the single biggest habit to break coming from another language. In Python the indentation IS the block structure, so getting it wrong changes the meaning of the program or stops it running.

**✗** It sets how far output is printed from the left edge

Wrong. Output position is decided by what you print, not by how the source is laid out.

**✓** It marks which statements belong to a block, and is part of the syntax

Correct. Where C uses braces, Python uses the indentation itself. An inconsistent indent raises `IndentationError`.

**✗** It is required only inside functions

Wrong. Every block needs it: `if`, `else`, `for`, `while`, `def`, `class`, `try`, `with`.

The answer is It marks which statements belong to a block, and is part of the syntax. Indentation is syntax, not decoration. It replaces the braces that other languages use to group statements.

Which of these is a **VALID** Python variable name?

- a. 2nd\_score
- b. class
- c. total score
- d. \_total2

**X** 2nd\_score

Wrong. A name may not start with a digit. Letters and the underscore only, then digits are allowed after the first character.

**X** class

Wrong. class is a reserved keyword, so it cannot be used as a name. The same applies to if, for, def, and the rest.

**X** total score

Wrong. A space separates two names, so Python reads this as two things and reports a syntax error. Use an underscore.

**✓** \_total2

Correct. It starts with an underscore, which is allowed, and the digit is not in first position.

The answer is \_total2. Start with a letter or underscore, then letters, digits or underscores, and never a keyword. Names are case sensitive, so Total and total are two different variables.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
print("A", "B")  
print("A" + "B")
```

a. A B

AB

b. AB

AB

c. A B

A B

d. A,B

AB

✓ A B

AB

Correct. A comma in print puts one space between the items; + joins two strings with nothing between.

✗ AB

AB

Wrong. The comma form adds a space between the two items.

✗ A B

A B

Wrong. + joins the strings end to end with no space.

✗ A,B

AB

Wrong. The comma separates the arguments; it is not printed.

`print('A', 'B')` prints its items separated by a single space. `'A' + 'B'` is one string, `AB`, before `print` ever sees it.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
t = 15
if t > 20:
    print("hot")
    print("stay inside")
print("done")
```

- a. hot  
    stay inside  
    done
- b. stay inside  
    done
- c. done
- d. hot  
    done

✗ hot  
    stay inside  
    done

Wrong. The indented lines only run when the condition is True, and  $15 > 20$  is False.

✗ stay inside  
    done

Wrong. Both indented lines belong to the if, so they are skipped together.

✓ done

Correct.  $t > 20$  is False, so both indented lines are skipped; `print('done')` is back at the margin and always runs.

**X** hot  
done

Wrong. Neither indented line runs; they belong to the same block.

Indentation decides which lines belong to the if. The two indented prints are the block; the unindented print is outside it.

**1.8**

**EASY**

Lesson 1.1

### What does it mean that Python is INTERPRETED?

- a. An interpreter reads and carries out the code line by line, rather than translating the whole program into machine code first
- b. The whole program must be compiled into machine code before any of it runs
- c. Every variable must be declared with its type before use
- d. Python code can only be run on the computer it was written on

**✓** An interpreter reads and carries out the code line by line, rather than translating the whole program into machine code first

Correct. That is the course's definition of an interpreted language.

**X** The whole program must be compiled into machine code before any of it runs

Wrong. That describes a COMPILED language.

**X** Every variable must be declared with its type before use

Wrong. Python is dynamically typed; no declarations are needed.

**X** Python code can only be run on the computer it was written on

Wrong. Being interpreted says nothing of that kind.

Interpreted means the interpreter executes the source line by line. That is why a runtime error on line 40 appears only after lines 1 to 39 have run.

1.9

EASY

Lesson 1.1

Which two of these lines must end with a colon?

- a. `if score >= 50`
- b. `total = total + 1`
- c. `def greet(name)`
- d. `print("done")`

✓ `if score >= 50`

Correct. An `if` opens a block, so its line ends with a colon.

✗ `total = total + 1`

Wrong. An assignment does not open a block.

✓ `def greet(name)`

Correct. A `def` opens a block, so its line ends with a colon.

✗ `print("done")`

Wrong. A `print` call does not open a block.

Every line that opens a block, `if`, `elif`, `else`, `for`, `while`, `def`, `try`, `except`, ends with a colon, and the block under it is indented.

1.10

EASY

Lesson 1.1

True or false: Python is case sensitive, so `Print("hi")` works exactly like `print("hi")`.

Answer: **false**

Case sensitive means `name`, `Name` and `NAME` are three different names. `print` exists; `Print` does not.

What are the three parts of almost every program on this paper, in order?

- a. Input, processing, output
- b. Output, input, processing
- c. Processing, output, input
- d. Compile, link, run

✓ Input, processing, output

Correct. Read something in, work something out, print the answer.

✗ Output, input, processing

Wrong. The output comes last.

✗ Processing, output, input

Wrong. You cannot process before you have the input.

✗ Compile, link, run

Wrong. Python is interpreted; there is no separate compile and link step to write.

Every long question is input, then processing, then output, however elaborate the story around it.

**Python is best described as which kind of language?**

- a. An interpreted, high-level, general-purpose language
- b. A compiled language, translated to machine code before it runs
- c. A markup language for describing documents
- d. An assembly language for a specific processor family

**✓ An interpreted, high-level, general-purpose language**

Correct. Interpreted means a program runs it line by line; high-level means it hides the machine; general-purpose means it is not tied to one kind of task.

**✗ A compiled language, translated to machine code before it runs**

Wrong. Compilation to machine code ahead of running is the C and Java model. Python translates and runs statement by statement through an interpreter, which is why an error can appear only when that line runs.

**✗ A markup language for describing documents**

Wrong. A markup language describes how content is structured, like HTML. Python computes, it does not mark up.

**✗ An assembly language for a specific processor family**

Wrong. Assembly is tied to one processor and its instruction set. Python code runs unchanged wherever the interpreter runs.

The answer is An interpreted, high-level, general-purpose language. Interpreted, high-level, general-purpose. Those three words are the definition the paper wants back.

Which of these is NOT a stated advantage of Python?

- a. It is easy to read, with a syntax close to English
- b. It is portable: the same code runs on different platforms
- c. It is the fastest executing language available
- d. It has a large standard library and a large community

✗ It is easy to read, with a syntax close to English

Wrong choice, because readability IS an advantage. The whitespace rules exist to force it.

✗ It is portable: the same code runs on different platforms

Wrong choice, because portability IS an advantage. The interpreter hides the platform.

✓ It is the fastest executing language available

Correct, and this is the one to remember. Interpretation costs speed. Python is chosen for programmer time, not for execution time, and a paper listing slow execution as a disadvantage of Python is common.

✗ It has a large standard library and a large community

Wrong choice, because the standard library IS an advantage. It is the "batteries included" claim.

The answer is It is the fastest executing language available. Speed of execution is a DISadvantage of Python, not an advantage. Everything else on that list is real.

At the interactive prompt you type `2 + 3` and press enter. Why does 5 appear even though you never wrote `print`?

- a. Python adds a `print` automatically to every line of every program
- b. The plus operator prints its result
- c. It does not appear; the prompt shows nothing without `print`
- d. The prompt echoes the value of any expression you type

✗ Python adds a `print` automatically to every line of every program

Wrong, and this is the trap. Inside a `SCRIPT`, `2 + 3` on its own line computes 5 and throws it away silently. Only the prompt echoes.

✗ The plus operator prints its result

Wrong. The plus operator produces a value and does nothing else with it.

✗ It does not appear; the prompt shows nothing without `print`

Wrong. The prompt does echo. Try it: this is the difference between the three kinds of code block this manual sets out.

✓ The prompt echoes the value of any expression you type

Correct. The interactive prompt evaluates what you type and displays the result. That is a feature of the prompt, not of the language.

The answer is The prompt echoes the value of any expression you type. The prompt echoes; a script does not. This is why the manual marks its three kinds of block differently, and why copying prompt examples into a file makes the output vanish.

[illegible]

### What does this print?

```
x = 4
y = x * 3
print(x)
x = 10
print(y)
```

- a.  $\frac{12}{4}$
- b.  $\frac{4}{12}$
- c.  $\frac{4}{10}$
- d.  $\frac{4}{30}$

~~X~~ 12  
4

Wrong. Lines run top to bottom, so x is printed before y.

✓ 4  
12

Correct. y is worked out as  $4 * 3 = 12$  when that line runs; changing x later does not go back and change y.

~~4~~ 4  
10

Wrong. The second print shows y, not the new x.

**X** 4  
30

Wrong. y was calculated when x was 4. Reassigning x afterwards does not recalculate y.

Python runs one line at a time, top to bottom, and each line uses the values that exist at that moment. y became 12 on line 2 and stays 12.

**1.16**

MEDIUM

Lesson 1.1

**The variable name on line 3 is misspelt. What happens when this runs?**

```
print("start")
total = 5
print(totl)
print("end")
```

- a. start is printed, then a NameError stops the program
- b. Nothing is printed; the NameError is found before anything runs
- c. start and end are both printed, with a warning
- d. start is printed, then end, and the misspelt line is skipped

**✓** start is printed, then a NameError stops the program

Correct. Python runs line by line: line 1 prints, line 3 raises NameError, and the program stops before line 4.

**X** Nothing is printed; the NameError is found before anything runs

Wrong. A misspelt NAME is only discovered when that line runs, so line 1 has already printed.

**X** start and end are both printed, with a warning

Wrong. An unhandled error stops the program; line 4 never runs.

**X** start is printed, then end, and the misspelt line is skipped

Wrong. Python does not skip a failing line; it stops there.

Python interprets as it goes, so everything before the faulty line runs normally. A name error is found only when the line using the name is reached.

## 2 Python Basics, Syntax, Operators and Strings

2.1

EASY

Manual O.2 Q1

What does it mean to say Python is dynamically typed?

- a. A variable must be declared with its type before use
- b. Types are checked by the compiler before the program runs
- c. Every value in Python is a string until converted
- d. The type belongs to the value, and a name may be rebound to a value of another type

**X** A variable must be declared with its type before use

Wrong, and this is the static-typing model of C and Java. Python needs no declaration at all: assignment creates the name.

**X** Types are checked by the compiler before the program runs

Wrong. There is no compiler pass checking types ahead of time. A type error surfaces when the offending line runs.

**X** Every value in Python is a string until converted

Wrong. This describes what `input()` returns, not what Python does generally. 5 is an int the moment you write it.

**✓** The type belongs to the value, and a name may be rebound to a value of another type

Correct. `x = 5` then `x = "five"` is legal, because the type travels with the value, not with the name.

The answer is The type belongs to the value, and a name may be rebound to a value of another type. The value carries the type; the name is only a label. That is why `type()` reports on the value and why a name can change type mid-program.

### What is meant by saying that Python strings are immutable?

- a. A string cannot be printed twice
- b. Once created, a string object cannot be changed; operations return a new string
- c. A string cannot be assigned to a variable more than once
- d. A string cannot contain numbers

✗ A string cannot be printed twice

Wrong. Printing does not change anything, and there is no limit on how often you may print.

✓ Once created, a string object cannot be changed; operations return a new string

Correct. `s.upper()` hands back a NEW string and leaves `s` untouched, and `s[0] = "X"` raises `TypeError`. To "change" a string you must rebind the name to a new one.

✗ A string cannot be assigned to a variable more than once

Wrong. Rebinding a name is always allowed. That changes what the name points to, not the string it used to point at.

✗ A string cannot contain numbers

Wrong. `"abc123"` is a perfectly ordinary string. The characters in it are irrelevant to mutability.

The answer is Once created, a string object cannot be changed; operations return a new string. Immutable means the object cannot be modified in place. This is a question the paper asks in words ("Strings are Immutable. Explain and with reasons."), so have the sentence ready and an example with it.

Which error does Python raise for `int("abc")`?

- a. `TypeError`
- b. `ValueError`
- c. `SyntaxError`
- d. `NameError`

**X** `TypeError`

Wrong. `TypeError` is for the wrong KIND of thing, such as `"5" + 5`. Here the kind is right, a string, and `int()` does accept strings.

**✓** `ValueError`

Correct. The type is acceptable but the VALUE is not: `"abc"` is a string that does not spell a number. That is exactly what `ValueError` means.

**X** `SyntaxError`

Wrong. The line parses perfectly. A syntax error would mean Python could not read the line at all.

**X** `NameError`

Wrong. `NameError` means a name was used before it was assigned. Every name here exists.

The answer is `ValueError`. `TypeError` is the wrong type, `ValueError` is the right type carrying an impossible value. `int("abc")` is the standard example of the second.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
print(int("12") + float("0.5"))
```

- a. 12.5
- b. 12
- c. an error
- d. "12"0.5

✓ 12.5

Correct. `int("12")` is 12 and `float("0.5")` is 0.5; an int plus a float gives a float.

✗ 12

Wrong. Adding 0.5 cannot give a whole number.

✗ an error

Wrong. Both conversions succeed.

✗ "12"0.5

Wrong. Both strings were converted to numbers, so + adds rather than joins.

Convert first, then calculate. Mixing an int and a float gives a float.

2.5

EASY

Lesson 1.2

A program reads a person's height in metres, such as 1.75. Which line is correct?

- a. `height = int(input("Height: "))`
- b. `height = input("Height: ")`
- c. `height = str(input("Height: "))`
- d. `height = float(input("Height: "))`

✗ `height = int(input("Height: "))`

Wrong. `int` would make 1.75 into 1, silently wrong.

✗ `height = input("Height: ")`

Wrong. `input` alone gives a string, which cannot be used in arithmetic.

✗ `height = str(input("Height: "))`

Wrong. `input` already returns a string; `str` changes nothing.

✓ `height = float(input("Height: "))`

Correct. A height has a fractional part, so it needs `float`.

| `int` for counts, `float` for measurements, `str` to turn a number back into text.

2.6

EASY

Lesson 1.2

True or false: `input()` returns an `int` when the user types only digits.

Answer: **false**

| The keyboard has no number key that Python believes: `input()` returns text, always.



**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
print(9 / 3)
```

- a. 3
- b. 3.0
- c. 3.00
- d. an error

✗ 3

Wrong. That would be `9 // 3`.

✓ 3.0

Correct. `/` always returns a float, even when the division is exact.

✗ 3.00

Wrong. Python prints the float as `3.0`.

✗ an error

Wrong. `9 / 3` is perfectly valid.

One slash, one point: `/` always gives a float.

2.9

EASY

Lesson 1.3

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
n = 38
print(n % 10, n // 10)
```

- a. 3 8
- b. 8 3
- c. 0 38
- d. 3.8 8

✗ 3 8

Wrong. The first value is the remainder, 8.

✓ 8 3

Correct.  $38 \% 10$  is the last digit, 8, and  $38 // 10$  is 3.

✗ 0 38

Wrong.  $38 \% 10$  is 8, not 0.

✗ 3.8 8

Wrong. `//` gives a whole number.

|  $n \% 10$  gives the last digit and  $n // 10$  drops it: a pair that turns up in digit questions.

2.10

EASY

Lesson 1.3

**True or false: = compares two values and == assigns a value.**

**Answer: false**

| = puts a value into a name; == asks whether two values are equal. Using = inside an if is a syntax error.



Which fault stops this program from running at all?

```
total = 0
for n in range(3)
    total += n
print(total)
```

- a. range(3) should be range(1, 3)
- b. total += n should be total = n
- c. The for line has no colon at the end
- d. print(total) should be indented

✗ range(3) should be range(1, 3)

Wrong. range(3) is valid; it gives 0, 1, 2.

✗ total += n should be total = n

Wrong. += is the correct way to accumulate.

✓ The for line has no colon at the end

Correct. A missing colon is a `SyntaxError`, found before anything runs.

✗ print(total) should be indented

Wrong. The print belongs after the loop, so it is correctly unindented.

! Sweep one is syntax: quotes, colons, commas, brackets. Every block-opening line needs its colon.

**Asked twice.** This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.2 and Lesson 3.4 and Lesson 3.5.

### What happens when this runs?

```
student_name = "Ada"  
print("Hi", Student_name)
```

- a. It prints Hi Ada
- b. `SyntaxError`, because of the capital letter
- c. It prints Hi `Student_name`
- d. `NameError`, because `Student_name` is not the same name as `student_name`

✗ It prints Hi Ada

Wrong. The two names differ in case, so Python treats them as different names.

✗ `SyntaxError`, because of the capital letter

Wrong. A capital letter in a name is legal; the name simply does not exist.

✗ It prints Hi `Student_name`

Wrong. Without quotes it is a name, not text.

✓ `NameError`, because `Student_name` is not the same name as `student_name`

Correct. Python is case sensitive, so `Student_name` was never defined.

Sweep two is names: check every name letter for letter and capital for capital against where it was defined.

### What is wrong with this program?

```
x = 5
if x = 5:
    print("five")
```

- a. = should be == in the if, and that is a syntax error
- b. print must not be indented
- c. x must be declared as an int first
- d. The if needs brackets around the condition

✓ = should be == in the if, and that is a syntax error

Correct. = assigns; a condition needs == to compare.

✗ print must not be indented

Wrong. The indented print is the if's block, as it should be.

✗ x must be declared as an int first

Wrong. Python needs no declarations.

✗ The if needs brackets around the condition

Wrong. Brackets are optional in Python.

Using = where == was meant is a planted fault; inside an if it will not even run.

**Asked twice.** This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.2 and Lesson 3.4 and Lesson 3.5.

### What happens when this runs?

```
msg = "Hello'  
print(msg)
```

- a. It prints Hello'
- b. A NameError, because msg is undefined
- c. A SyntaxError, because the string opens with a double quote and closes with a single quote
- d. It prints Hello

**X** It prints Hello'

Wrong. The single quote does not close a double-quoted string.

**X** A NameError, because msg is undefined

Wrong. The fault is found before any name is used.

**✓** A SyntaxError, because the string opens with a double quote and closes with a single quote

Correct. The string never closes, so Python cannot read the line.

**X** It prints Hello

Wrong. The line cannot be parsed, so nothing runs.

Line 1 of the paper's broken program had exactly this: "Serve Jesus Now'.

**In what order does the checklist sweep a broken listing?**

- a. All at once, in a single careful read
- b. Meaning, then names, then syntax
- c. Names, then syntax, then meaning
- d. Syntax, then names, then meaning

**✗ All at once, in a single careful read**

Wrong. One pass looking for everything finds one or two errors, not all of them.

**✗ Meaning, then names, then syntax**

Wrong. Syntax comes first, because nothing runs until it is fixed.

**✗ Names, then syntax, then meaning**

Wrong. Syntax comes before names.

**✓ Syntax, then names, then meaning**

Correct. First what stops it running, then names checked against their definitions, then indexes, types and logic.

**Three sweeps, and count your findings against the marks before you stop.**

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
print(2 + 3 * 4 ** 2)
```

- a. 80
- b. 400
- c. 50
- d. 100

✗ 80

Wrong. That is  $(2 + 3) * 4 ** 2$ , which adds before multiplying. Addition is last, not first.

✗ 400

Wrong. That is  $((2 + 3) * 4) ** 2$ , which applies the exponent to everything.

✓ 50

Correct. Exponent first:  $4 ** 2$  is 16. Then multiply:  $3 * 16$  is 48. Then add:  $2 + 48$  is 50.

✗ 100

Wrong. That is  $(2 + 3 * 4) ** 2$ , exponent applied last.

The answer is 50. Exponent, then multiply and divide, then add and subtract. When in doubt in the exam, write the brackets in yourself: they cost nothing and remove the question.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
print(2 ** 3 ** 2)
```

- a. 64
- b. 18
- c. 12
- d. 512

**X** 64

Wrong. 64 is  $(2 ** 3) ** 2$ , grouping from the left. The exponent operator is the exception: it groups from the RIGHT.

**X** 18

Wrong. 18 would be  $2 * 3 * 3$ , which is not what any part of this expression does.

**X** 12

Wrong. There is no reading of this expression that gives 12.

**✓** 512

Correct. It reads as  $2 ** (3 ** 2)$ , which is  $2 ** 9$ , which is 512.

The answer is 512. Exponent is right-associative, and it is the only common operator that is. Everything else in this course groups left to right.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
x = 10
x += 5
x -= 3
x *= 2
print(x)
```

a. 24

b. 12

c. 30

d. 17

✓ 24

Correct. 10 plus 5 is 15, minus 3 is 12, times 2 is 24. Each line uses the value the line before it left behind.

✗ 12

Wrong. 12 is the value after the subtraction. The multiplication still has to happen.

✗ 30

Wrong. 30 would be 15 times 2, which skips the subtraction.

✗ 17

Wrong. 17 would be 10 plus 5 plus 2, which misreads two of the three operators.

The answer is 24.  $x += 5$  is exactly  $x = x + 5$ . Work these line by line and write the running value in the margin; the exam asks this shape often.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
s = "PYTHON"  
print(s[0], s[-1], s[1:4], s[::-1])
```

- a. P N YTHO NOHTYP
- b. Y N YTH NOHTYP
- c. P N YTH NOHTYP
- d. P O YTH NOHTYP

**X** P N YTHO NOHTYP

Wrong. The slice `s[1:4]` does not include index 4, so it is three characters, not four.

**X** Y N YTH NOHTYP

Wrong. `s[0]` is P. Python indexes from 0, not from 1.

**✓** P N YTH NOHTYP

Correct. Index 0 is the first character, minus 1 is the last, a slice stops BEFORE its end index so `1:4` gives characters 1, 2 and 3, and a step of minus 1 walks the string backwards.

**X** P O YTH NOHTYP

Wrong. `s[-1]` is the last character, N, not the second to last.

The answer is P N YTH NOHTYP. Index from 0, negative counts from the end, a slice excludes its stop, and `[::-1]` reverses. Those four facts answer most string questions on this paper.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
name = "Ada"  
score = 87.456  
print(f"{name} scored {score:.1f}")
```

- a. Ada scored 87.456
- b. Ada scored 87.5
- c. Ada scored 87.4
- d. {name} scored {score:.1f}

✗ Ada scored 87.456

Wrong. The .1f asks for one decimal place, so the full value is not printed.

✓ Ada scored 87.5

Correct. The f before the quote turns it into a formatted string, the braces are replaced by the values, and .1f rounds to one decimal place, giving 87.5.

✗ Ada scored 87.4

Wrong. Formatting ROUNDS, it does not truncate, and 87.456 rounds up to 87.5.

✗ {name} scored {score:.1f}

Wrong. That is what you get WITHOUT the f prefix, which is a real and common slip: the braces then print literally.

The answer is Ada scored 87.5. f"..." substitutes; :.2f fixes the decimal places. Forgetting the f is the classic error and the output tells you at once.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
a, b = 1, 2
a, b = b, a
print(a, b)
```

a. 2 1

b. 1 2

c. 2 2

d. 1 1

✓ 2 1

Correct. The right-hand side is evaluated first, into the pair (2, 1), and only then unpacked into a and b. So the swap needs no temporary variable.

✗ 1 2

Wrong. That is the value before the swap.

✗ 2 2

Wrong. That is what happens in a language where `a = b` runs first and destroys the old a. Python does not: it builds the whole right side first.

✗ 1 1

Wrong, for the same reason: nothing is overwritten before it is read.

The answer is 2 1. `a, b = b, a` swaps in one line because the right side is built completely before anything is assigned.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
print(type(3.0).__name__, type("3").__name__, type(3 > 2).__name__)
```

- a. float float bool
- b. float str str
- c. int str bool
- d. float str bool

**X** float float bool

Wrong. "3" is in quotes, so it is a str.

**X** float str str

Wrong.  $3 > 2$  is a comparison, which gives True, a bool.

**X** int str bool

Wrong. 3.0 is a float, not an int, because of its decimal point.

**✓** float str bool

Correct. 3.0 has a decimal point, "3" is in quotes, and a comparison gives a bool.

The decimal point makes a float; quotes make a str; a comparison makes a bool. `type(x).__name__` gives the plain name.

The user types 5 at the prompt, so a holds "5". What does this print?

```
a = "5"      # what input() handed back  
b = 2  
print(a * b)
```

- a. 7
- b. 10
- c. 55
- d. an error, because text cannot be multiplied

✗ 7

Wrong. Nothing here adds 5 and 2.

✗ 10

Wrong. That would need `int(a) * 2`. a is text.

✓ 55

Correct. a is the string "5", and \* on a string repeats it: "5" twice is 55.

✗ an error, because text cannot be multiplied

Wrong. A string times a whole number is allowed; it repeats the string.

input() always returns a string. "5" \* 2 repeats the text, which is the silent version of the conversion mistake: it runs and gives the wrong answer.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
x = 7
print(type(x).__name__)
x = "seven"
print(type(x).__name__)
```

- a. int  
str
- b. str  
str
- c. int  
int
- d. an error, because x changed type

✓ int  
str

Correct. The type belongs to the value. x first holds an int, then is rebound to a str.

✗ str  
str

Wrong. At the first print x holds the whole number 7.

✗ int  
int

Wrong. After the reassignment x holds text.

✗ an error, because x changed type

Wrong. Python is dynamically typed, so a name may be rebound to any type.

Dynamically typed means the type belongs to the value, not the name, so no declaration is needed and a name can be rebound to a different type.

2.26

MEDIUM

Lesson 1.2

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
n = int("3") * 2
s = str(n) + "0"
print(s, type(s).__name__)
```

- a. 6 str
- b. 60 str
- c. 60 int
- d. 12 str

✗ 6 str

Wrong. The "0" is joined on the end.

✓ 60 str

Correct.  $n$  is  $3 * 2 = 6$ ; `str(6) + "0"` is the text "60", a str.

✗ 60 int

Wrong. Joining with `+` on strings gives a str, not an int.

✗ 12 str

Wrong.  $n$  is 6, not 12.

`str()` turns a number into text so it can be joined; the result is text, even when it looks like a number.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
print(3 + 2 ** 2 * 2)
```

- a. 11
- b. 14
- c. 18
- d. 50

✓ 11

Correct. Power first,  $2 ** 2 = 4$ ; then  $4 * 2 = 8$ ; then  $3 + 8 = 11$ .

✗ 14

Wrong. That squares  $2 * 2$  in the wrong order.

✗ 18

Wrong. That adds  $3 + 2$  first.

✗ 50

Wrong. That treats the whole thing as  $(3 + 2) ** 2 * 2$ .

■ **\*\*** is done first of all, then **\***, then **+**.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
x = 3
print(x > 1 and x < 3, x > 1 or x > 5, not x == 3)
```

- a. False True False
- b. True False True
- c. True True False
- d. False False True

✓ False True False

Correct.  $x < 3$  is False so the and is False;  $x > 1$  is True so the or is True;  $x == 3$  is True so not gives False.

✗ True False True

Wrong. Check each part: and needs both, or needs one, not flips.

✗ True True False

Wrong.  $3 < 3$  is False, so the and is False.

✗ False False True

Wrong. The or needs only one True side, and  $x > 1$  is True.

and is True only when both sides are True; or when at least one is; not flips it.

[illegible]

### What does this print?

```
total = 4
total *= 3
total -= 2
total /= 2
print(total)
```

- a. 5  
b. 10.0  
c. 5.0  
d. 4.0

**X** 5

Wrong. `/=` uses true division, which always gives a float.

**X** 10.0

Wrong. The last step halves 10.

✓ 5.0

Correct.  $4 * 3 = 12$ , minus  $2 = 10$ , then  $/ = 2$  gives  $5.0$ , a float because of the single slash.

**X** 4.0

Wrong. Work each line in order: 12, 10, 5.0.

total += 5 means total = total + 5, and likewise for the others. /= divides with /, so the result is a float.



**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
s = "banana"
print(s.count("a"), s.find("n"), s.find("z"))
```

- a. 3 3 -1
- b. 2 2 -1
- c. 3 2 None
- d. 3 2 -1

✗ 3 3 -1

Wrong. find gives the FIRST n, at index 2.

✗ 2 2 -1

Wrong. Count them: b-a-n-a-n-a has three a's.

✗ 3 2 None

Wrong. find returns -1, not None, when nothing is found.

✓ 3 2 -1

Correct. banana has three a's; the first n is at index 2; find gives -1 when the text is absent.

count(x) counts occurrences; find(x) gives the index of the first match, or -1.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
s = "hello"  
s.upper()  
print(s)
```

- a. None
- b. Hello
- c. HELLO
- d. hello

**X** None

Wrong. s still holds its original string.

**X** Hello

Wrong. Nothing here capitalises one letter.

**X** HELLO

Wrong. The upper-case copy was never assigned to anything.

**✓** hello

Correct. upper() returns a new string, which was thrown away; s itself never changes.

Strings are immutable. A string method hands back a NEW string; to keep it you must assign it, s = s.upper().

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
w = " data "  
print(len(w), len(w.strip()))
```

a. 4 4

b. 8 8

c. 8 4

d. 6 4

✗ 4 4

Wrong. len counts the spaces too.

✗ 8 8

Wrong. strip() removes the spaces at both ends.

✓ 8 4

Correct. The string has two spaces, four letters and two spaces: 8 characters; strip() removes the spaces, leaving 4.

✗ 6 4

Wrong. Count the spaces on both sides: two and two.

len counts every character, spaces included. strip() returns a copy without the whitespace at the ends.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
words = "red green blue".split()
print(len(words))
print("-".join(words))
```

- a. 14  
red-green-blue
- b. 3  
red green blue
- c. 1  
red-green-blue
- d. 3  
red-green-blue

**X** 14  
red-green-blue

Wrong. `len(words)` counts the list's items, not the characters.

**X** 3  
red green blue

Wrong. `join` puts the separator it is called on, the dash, between the words.

**X** 1  
red-green-blue

Wrong. `split()` gives a list of three separate words.

**✓** 3  
red-green-blue

Correct. `split()` cuts on spaces into a list of three words, and `"-".join` glues them back with dashes.

| split() returns a list; sep.join(list) returns a string, with sep between the items.

2.35

MEDIUM

Lesson 1.4

**Asked twice.** This same question also appears as: Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.2 and Lesson 3.4 and Lesson 3.5.

**What happens when this runs?**

```
print("abc".index("d"))
```

- a. It prints 0
- b. It prints -1
- c. It prints None
- d. It raises ValueError, because index() raises when the text is absent

✗ It prints 0

Wrong. "d" is not in "abc" at all.

✗ It prints -1

Wrong. That is what find() would return.

✗ It prints None

Wrong. Neither method returns None.

✓ It raises ValueError, because index() raises when the text is absent

Correct. index() does the same job as find() but raises ValueError on a miss.

| Use find() when a miss is expected: it returns -1. index() raises ValueError instead.

2.36

MEDIUM

Lesson 1.4

**True or false: s[-0] gives the last character of s.**

Answer: **false**

| The paper's broken program used my\_string[-0] for the last character. It is the first; the last is -1.

**Asked twice.** This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.2 and Lesson 3.4 and Lesson 3.5.

### What happens when this runs?

```
person = {"name": "Ada"}  
print(person["Name"])
```

- a. It raises `KeyError: 'Name'`, and the program stops
- b. It prints Name
- c. It prints None
- d. It prints Ada

✓ It raises `KeyError: 'Name'`, and the program stops

Correct. Keys are case sensitive; 'Name' is not a key, so the lookup raises `KeyError`.

✗ It prints Name

Wrong. The lookup fails; nothing is printed.

✗ It prints None

Wrong. `d[key]` raises; only `d.get(key)` returns None.

✗ It prints Ada

Wrong. The key stored is 'name', lower case.

A dictionary key is a string, so it is case sensitive, and a missing key stops the program with `KeyError`.

Which fault is in this line, and what kind of error is it?

```
total = 5  
print("Total:" total)
```

- a. A missing comma between the two items: a `SyntaxError`
- b. A missing colon: a `SyntaxError`
- c. Text cannot be printed with a number: a `TypeError`
- d. total is undefined: a `NameError`

✓ A missing comma between the two items: a `SyntaxError`

Correct. The two things being printed need a comma between them.

✗ A missing colon: a `SyntaxError`

Wrong. A print call does not open a block, so it needs no colon.

✗ Text cannot be printed with a number: a `TypeError`

Wrong. print can show text and numbers together, separated by a comma.

✗ total is undefined: a `NameError`

Wrong. total is defined on the first line; the problem is the missing comma.

Missing commas and colons are the syntax faults the examiner plants most often.

The programmer wanted the LAST character. What does this print?

```
s = "Python"  
last = s[-0]  
print(last)
```

- a. an error
- b. o
- c. n
- d. P

**X** an error

Wrong. It runs; it just gives the wrong character.

**X** o

Wrong. o is at index 4.

**X** n

Wrong. That is what s[-1] would give.

**✓** P

Correct. -0 is 0, so s[-0] is the first character, P.

Sweep three is meaning: the program runs and quietly gives the wrong answer. The last character is s[-1].

The user typed 20, so age holds "20". What happens next?

```
age = "20"      # what input() handed back  
print(age + 1)
```

- a. It prints 20 1
- b. TypeError, because text and a number cannot be added
- c. It prints 21
- d. It prints 201

✗ It prints 20 1

Wrong. + does not insert a space; it fails here.

✓ TypeError, because text and a number cannot be added

Correct. "20" + 1 mixes a str and an int, which + refuses.

✗ It prints 21

Wrong. age was never converted with int().

✗ It prints 201

Wrong. That would need str(1); + will not join text to a number.

The fix is `age = int(input("Age: "))`. Unconverted input is the commonest planted type error.

A program does `age = input()` and the user types 20. What is printed?

```
age = "20"          # this is exactly what input() hands back
print(type(age).__name__)
print(age * 2)
```

- a. str  
2020
- b. int  
40
- c. str  
40
- d. int  
2020

✓ str  
2020

Correct. `input()` always returns a str, so `type` reports str, and `"20" * 2` repeats the characters to give 2020.

✗ int  
40

Wrong on both lines. `input()` never returns an int, and star on a string repeats rather than multiplies.

✗ str  
40

Wrong on the second line. 40 would need age to be a number; it is a string, so star repeats it.

✗ int  
2020

Wrong on the first line. The value came from `input()`, so it is a string however numeric it looks.

The answer is str then 2020. `input()` always returns a string. That is why `age + 5` raises `TypeError` and `age * 2` gives 2020, and it is the single most common first-program bug in this course. Wrap it in `int()` or `float()`.

2.42

HARD

Manual O.2 Q3

**Asked twice.** This same question also appears as: Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
print(7 / 2)
print(7 // 2)
print(7 % 2)
```

a. 3

3

1

b. 3.5

3

1

c. 3.5

3.5

1

d. 3.5

3

3.5

**X** 3

3

1

Wrong. A single slash never truncates in Python 3.  $7 / 2$  is 3.5, not 3. That was Python 2 behaviour, and the change is a favourite exam point.

**✓** 3.5

3

1

Correct. A single slash is true division and always gives a float, even when it divides exactly. Double slash floors the result to an integer. The percent sign gives the remainder.

**X** 3.5

3.5

1

Wrong. Double slash is floor division, so it produces 3, not 3.5.

**X** 3.5

3

3.5

Wrong. The percent sign is remainder, not division, so  $7 \% 2$  is 1.

The answer is 3.5 then 3 then 1. Slash gives a float, double slash floors, percent is the remainder.  
Note  $7 // 2$  is 3 but  $-7 // 2$  is -4, because flooring goes DOWN, not towards zero.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
x = 5
print(1 < x < 10)
print(x == 5 and x != 5)
```

- a. True  
True
- b. True  
False
- c. False  
False
- d. TypeError

**X** True  
True

Wrong. The second line cannot be True:  $x == 5$  and  $x != 5$  contradict each other by construction.

**✓** True  
False

Correct. Python allows a chained comparison, and  $1 < 5 < 10$  is True. The second line asks for something and its own opposite at once, which can never be True.

**X** False  
False

Wrong. The first line is True, because chaining is legal in Python and 5 does lie between 1 and 10.

**X** TypeError

Wrong. Nothing here is a type error; all the operands are numbers.

The answer is True then False. Chained comparison is real Python and reads exactly as it does in mathematics. Most languages do not have it, so it is worth a mark when the paper shows one.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
print(True and False or True)
print(not True and False)
```

a. False

False

b. True

True

c. True

False

d. False

True

✗ False

False

Wrong. The first line is True, because the or rescues it: only one side of an or needs to be true.

✗ True

True

Wrong. The second line is False. not True is False, and False and anything is False.

✓ True

False

Correct. and binds tighter than or, so the first line is (True and False) or True, which is False or True, which is True. On the second, not binds tightest, so it is (not True) and False, which is False and False.

✗ False

True

Wrong on both counts, and it has the precedence backwards.

The answer is True then False. Precedence among the three: not, then and, then or. Read them in that order and these questions become mechanical.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
print(int("7") + int(3.9))  
print(float("2.5") * 2)  
print(str(4) + "2")
```

a. 10.9

5.0

6

b. 11

5.0

42

c. 10

5

42

d. 10

5.0

42

**X** 10.9

5.0

6

Wrong. `int()` applied to a float does not keep the fraction. It truncates towards zero.

**X** 11

5.0

42

Wrong. `int(3.9)` is 3, not 4. `int()` truncates, it does not round; `round()` is the function that rounds.

**X** 10

5

42

Wrong. Multiplying a float by an int gives a float, so it prints 5.0, with the point.

✓ 10  
5.0  
42

Correct. `int("7")` is 7 and `int(3.9)` TRUNCATES to 3, so the sum is 10. `float("2.5") * 2` is 5.0, a float because one operand is. `str(4) + "2"` joins two strings into "42".

The answer is 10 then 5.0 then 42. `int()` truncates towards zero and `round()` rounds. Mixing an int with a float gives a float. Plus on two strings joins them.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
s = "  Hello World  "
print(s.strip().upper())
print(s.strip().split())
print("-".join(["a", "b", "c"]))
```

a. HELLO WORLD

['Hello World']

a-b-c

b. HELLO WORLD

['Hello', 'World']

a-b-c

c. Hello World

['Hello', 'World']

abc

d. HELLO WORLD

['Hello', 'World']

a-b-c

**X** HELLO WORLD

['Hello World']

a-b-c

Wrong. `split()` with no argument splits on EVERY run of whitespace, so two words come back as two items.

**X** HELLO WORLD

['Hello', 'World']

a-b-c

Wrong. `strip()` really does remove the outer spaces. That is its only job.

**X** Hello World

['Hello', 'World']

abc

Wrong on two counts: `upper()` does uppercase, and `join()` really does put the hyphen between the items.

✓ HELLO WORLD

`['Hello', 'World']`

a-b-c

Correct. `strip()` removes the leading and trailing spaces, `upper()` uppercases what is left, `split()` with no argument splits on whitespace into a list of words, and `join()` glues a list together with the string it is called on as the separator.

The answer is HELLO WORLD then `['Hello', 'World']` then a-b-c. `strip`, `upper`, `lower`, `split`, `join`, `replace`, `find`. Those seven cover nearly every string question this course has ever set.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
print("ab" * 3)
print("ab" + "3")
```

- a. ababab  
ab3
- b. ab3  
ababab
- c. abababab  
ab3
- d. TypeError  
ab3

✓ ababab  
ab3

Correct. A string times an integer repeats it. A string plus a string joins them. Both operators are overloaded for strings.

✗ ab3  
ababab

Wrong, and it has the two operators swapped. Star repeats, plus joins.

✗ abababab  
ab3

Wrong. Times 3 gives three copies, not four.

✗ TypeError  
ab3

Wrong. String times int is perfectly legal. It is string PLUS int that raises TypeError.

The answer is ababab then ab3. Star repeats, plus joins, and "ab" + 3 is the error. The operator means different things for numbers and for strings, which is what operator overloading is.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
print("a\tb")
print("c\nnd")
print("e\\f")
```

a. a\tb  
c\nnd  
e\\f

b. ab  
cd  
ef

c. a b  
c  
d  
e\f

d. a b  
cd  
ef

**X** a\tb  
c\nnd  
e\\f

Wrong. That is what a RAW string would print, written `r"a\tb"`. Without the `r`, the escapes are interpreted.

**X** ab  
cd  
ef

Wrong. The escapes produce real whitespace: a tab and a line break, not nothing.

**✓** a b  
c

```
d
e\f
```

Correct. Backslash t is a tab, backslash n is a newline, and two backslashes print one literal backslash.

```
X a b
  cd
  ef
```

Wrong. Backslash t is a tab character, which is wider than a single space, and the last line keeps its backslash.

The answer is a b then c then d then e\f. Backslash n newline, backslash t tab, double backslash for a real backslash. This matters for file paths, where a Windows path needs doubling or a raw string.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
a = [1, 2]
b = [1, 2]
print(a == b)
print(a is b)
```

a. True

True

b. False

False

c. False

True

d. True

False

✗ True

True

Wrong. `is` asks whether they are the SAME object, and two separately written lists are two objects however alike they look.

✗ False

False

Wrong. `==` on two lists compares them item by item, and these match.

✗ False

True

Wrong, and it has the two operators exactly backwards.

✓ True

False

Correct. `==` compares contents, so it is True. `is` compares identity, so it is False: `a` and `b` name two different list objects.

The answer is True then False. == is equal in value, is is the very same object. Use is only for None, and use == for everything else.

2.50 (HARD) Lesson 1.2

[illegible]

## What does this print?

```
print(int(-3.7))
```

- a. 4
- b. -3
- c. -4
- d. -3.7

**X 4**

Wrong. The sign is kept.

✓ -3

Correct. `int()` chops off the decimal part, towards zero, so -3.7 becomes -3.

**X** -4

Wrong. `int()` does not round down; it simply drops the fraction.

**X** -3.7

Wrong. `int()` returns a whole number.

`int()` truncates: it removes the fractional part and does not round. `int(7.9)` is 7 and `int(-3.7)` is -3.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
print("9" > "10")
```

- a. an error, because strings cannot be compared
- b. 10
- c. True
- d. False

**X** an error, because strings cannot be compared

Wrong. Strings can be compared; they compare in dictionary order.

**X** 10

Wrong. A comparison gives a bool.

**✓** True

Correct. Text compares character by character, and "9" comes after "1".

**X** False

Wrong. That would be true of the NUMBERS 9 and 10, but these are strings.

**|** This is the silent trap: comparing unconverted input compares text, so "9" > "10" is True.

## Lesson 1.3

[illegible]

### What does this print?

```
print(10 - 4 / 2 * 3)
```

- a. 1.0  
b. 4.0  
c. 9.0  
d. 4

**X** 1.0

Wrong. That multiplies  $2 * 3$  before dividing, but  $/$  and  $*$  go left to right.

✓ 4.0

Correct. / and \* first, left to right:  $4 / 2 = 2.0$ , then  $2.0 * 3 = 6.0$ , then  $10 - 6.0 = 4.0$ .

X 9.0

Wrong. That does the subtraction first, which is the wrong order.

X 4

Wrong. / made a float, so the answer is 4.0.

Precedence: \*\* first, then \* / // %, left to right, then + and -.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
def check():  
    print("checked")  
    return True  
  
print(False and check())  
print(True or check())
```

a. checked

False

checked

True

b. False

checked

True

c. checked

False

True

d. False

True

**X** checked

False

checked

True

Wrong. Neither call runs, because both expressions short circuit.

**X** False

checked

True

Wrong. True or ... already knows it is True, so the right side is skipped.

✗ checked

False

True

Wrong. False and ... already knows it is False, so check() is never called.

✓ False

True

Correct. and stops at a False left side and or stops at a True left side, so check() is never called.

and and or short circuit: if the left side settles the answer, the right side is never evaluated.



2.55

HARD

Lesson 1.4

**Asked twice.** This same question also appears as: Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.2 and Lesson 3.4 and Lesson 3.5.

**What happens when this runs?**

```
print("Hello"[5])
```

- a. It raises IndexError, because the last index of a five-letter string is 4
- b. It prints o
- c. It prints H
- d. It prints an empty string

✓ It raises IndexError, because the last index of a five-letter string is 4

Correct. Indexes run 0 to 4, so index 5 is out of range.

✗ It prints o

Wrong. o is at index 4.

✗ It prints H

Wrong. H is at index 0.

✗ It prints an empty string

Wrong. A SLICE past the end gives an empty string; a single INDEX past the end raises.

■ A slice that starts past the end quietly gives "", but a single index past the end is an IndexError.

### 3 Control Flow, Lists, Tuples, Sets and Dictionaries

3.1

EASY

Manual O.3 Q10

Which statement about tuples is TRUE?

- a. A tuple is ordered and immutable, and can be used as a dictionary key
- b. A tuple can be changed after creation, like a list
- c. A tuple cannot contain more than two items
- d. A tuple is written with square brackets

✓ A tuple is ordered and immutable, and can be used as a dictionary key

Correct. Ordered like a list, immutable unlike one, and because it cannot change it can be hashed, so it may be a dictionary key where a list may not.

✗ A tuple can be changed after creation, like a list

Wrong. That is a list. Assigning to `t[0]` on a tuple raises `TypeError`.

✗ A tuple cannot contain more than two items

Wrong. The name comes from the general mathematical term, not from two. A tuple can hold any number of items, including none.

✗ A tuple is written with square brackets

Wrong. Round brackets, or in fact just commas: `1, 2` is already a tuple. Square brackets make a list.

The answer is A tuple is ordered and immutable, and can be used as a dictionary key. List against tuple is a guaranteed question. Ordered both, mutable only the list, and only the tuple can be a dictionary key.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
score = 50
if score >= 70:
    grade = "A"
elif score >= 60:
    grade = "B"
elif score >= 50:
    grade = "C"
elif score >= 45:
    grade = "D"
else:
    grade = "F"
print("Grade:", grade)
```

- a. Grade: D
- b. Grade: F
- c. Grade: C
- d. Grade: B

**X** Grade: D

Wrong. The chain stops at the first True test, which is  $\geq 50$ .

**X** Grade: F

Wrong. else is reached only if every test fails.

**✓** Grade: C

Correct. 50 fails  $\geq 70$  and  $\geq 60$ , then matches  $\geq 50$ , and the chain stops.

**X** Grade: B

Wrong. 50 is not  $\geq 60$ .

In an if / elif chain exactly one block runs: the first whose condition is True.

3.3

EASY

Lesson 2.1

The same test written with elif. What does this print?

```
x = 15
if x > 10:
    print("big")
elif x > 5:
    print("medium")
else:
    print("small")
```

a. medium

b. big

c. big

medium

d. small

✗ medium

Wrong.  $x > 10$  is tested first and is True.

✓ big

Correct. The first True branch runs and the rest of the chain is skipped.

✗ big

medium

Wrong. elif is part of one chain, so only one branch runs.

✗ small

Wrong. else runs only when every test fails.

elif is a queue, if is a crowd: in a chain, only the first True branch runs.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
score = 105
if 0 <= score <= 100:
    print("ok")
else:
    print("invalid")
```

- a. ok  
invalid
- b. invalid
- c. an error, because comparisons cannot be chained
- d. ok

✗ ok

invalid

Wrong. if and else never both run.

✓ invalid

Correct.  $0 \leq 105 \leq 100$  is False because 105 is above 100.

✗ an error, because comparisons cannot be chained

Wrong. Python allows  $0 \leq \text{score} \leq 100$ .

✗ ok

Wrong. 105 is outside the range.

! A chained comparison is the neatest way to test a range: good when at least 0 AND at most 100.

**3.5****EASY**

Lesson 2.1

for and while loops belong to which of the three control structures?

- a. Decomposition
- b. Iteration
- c. Selection
- d. Sequence

**X** Decomposition

Wrong. That is not one of the three control structures.

**✓** Iteration

Correct. Iteration repeats a block while a condition holds or for each item.

**X** Selection

Wrong. Selection is if, elif and else.

**X** Sequence

Wrong. Sequence is statements one after another.

| Sequence, selection, iteration: one after another, choose one, do it again.

**3.6****EASY**

Lesson 2.1

True or false: an else can carry its own condition, as in else score < 50:

Answer: **false**

| Writing a condition after else is a planted syntax error.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
for i in range(2, 10, 3):  
    print(i, end=" ")
```

a. 3 6 9

b. 2 5 8 11

c. 2 3 4 5 6 7 8 9

d. 2 5 8

✗ 3 6 9

Wrong. The sequence starts at 2.

✗ 2 5 8 11

Wrong. 11 is beyond the stop of 10.

✗ 2 3 4 5 6 7 8 9

Wrong. The third number is the step, 3.

✓ 2 5 8

Correct. Start at 2, step 3, stop before 10: 2, 5, 8.

`range(start, stop, step)` starts at start and stops BEFORE stop. `end=" "` keeps the output on one line.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
total = 0
for n in [4, 7, 1]:
    total += n
print(total)
```

- a. 4
- b. 12
- c. 1
- d. 0

✗ 4

Wrong. The loop adds all three numbers.

✓ 12

Correct. total starts at 0 before the loop and adds 4, 7 and 1.

✗ 1

Wrong. That is what happens if total is reset inside the loop.

✗ 0

Wrong. The additions happen inside the loop.

Zero it before, add inside, print after: the accumulator pattern.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
n = 3
while n > 0:
    print(n)
    n -= 1
print("go")
```

a. 3  
2  
1  
go

b. 3  
2  
1  
0  
go

c. go

d. 2  
1  
0  
go

✓ 3  
2  
1  
go

Correct. n prints then drops by 1 each pass; when n reaches 0 the test fails and the loop ends.

X 3  
2  
1  
0  
go

Wrong. When  $n$  is 0 the condition  $n > 0$  is False, so 0 is not printed.

X go

Wrong.  $n > 0$  is True at the start, so the body runs.

X 2  
1  
0  
go

Wrong. The print comes before the decrease.

A while loop repeats while its condition is True; something in the body must eventually make it False.

### What is wrong with this loop?

```
n = 1
while n < 5:
    print(n)
```

- a. n never changes, so  $n < 5$  stays True and it loops forever
- b. The print should come after the loop
- c. while cannot compare numbers
- d. range() is missing

✓ n never changes, so  $n < 5$  stays True and it loops forever

Correct. Nothing in the body updates  $n$ , so the condition never becomes False.

✗ The print should come after the loop

Wrong. Moving the print does not fix the missing update.

✗ while cannot compare numbers

Wrong. while works with any condition.

✗ range() is missing

Wrong. A while loop does not use range.

Forgetting to update the variable in the condition is how an infinite loop is written by accident.

### What is a count controlled loop?

- a. A loop that never ends
- b. A loop that counts how many errors occurred
- c. A loop that repeats a fixed, known number of times, decided before the loop begins
- d. A loop that runs until the user types a sentinel

✗ A loop that never ends

Wrong. That is an infinite loop.

✗ A loop that counts how many errors occurred

Wrong. Not the definition.

✓ A loop that repeats a fixed, known number of times, decided before the loop begins

Correct. That is the definition, written in Python with `for` and usually `range()`.

✗ A loop that runs until the user types a sentinel

Wrong. That is a sentinel, or condition, controlled loop.

for when you know how many times; while when you are waiting for something to happen.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
d = {"a": 1}
print(d.get("b"), d.get("a"))
```

- a. b 1
- b. 0 1
- c. None 1
- d. KeyError

**X** b 1

Wrong. `get()` returns the value, not the key.

**X** 0 1

Wrong. `get()` returns `None`, not 0, unless told otherwise.

**✓** None 1

Correct. `get()` returns `None` for a missing key instead of raising, and the value for a present one.

**X** KeyError

Wrong. `d["b"]` would raise; `get()` does not.

**|** `d.get(key)` is the safe lookup: the value, or `None`. `d[key]` raises `KeyError`.

Each student's matric number is paired with that student's name. Which container suits this?

- a. A tuple, because the data is fixed
- b. A list, because it is ordered
- c. A dictionary, because each name is paired with its number and can be looked up by it
- d. A set, because the numbers are unique

✗ A tuple, because the data is fixed

Wrong. A tuple is one fixed record, not a lookup of many pairs.

✗ A list, because it is ordered

Wrong. A list would make you search for each pair.

✓ A dictionary, because each name is paired with its number and can be looked up by it

Correct. Paired data is key to value, which is exactly a dictionary.

✗ A set, because the numbers are unique

Wrong. A set holds single values, not pairs.

List grows, tuple is fixed, set is unique, dictionary pairs. Give the reason: the reason carries the mark.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
xs = [3, 1, 2]
ys = xs.sort()
print(ys)
```

- a. None
- b. [1, 2, 3]
- c. [3, 1, 2]
- d. TypeError

✓ None

Correct. `sort()` returns `None`. If you want a new sorted list without touching the original, use `sorted(xs)`.

✗ [1, 2, 3]

Wrong, and this is the trap. `sort()` changes `xs` and hands back nothing, so the sorted list is in `xs`, not in `ys`.

✗ [3, 1, 2]

Wrong. `xs` itself IS sorted now; it is `ys` that is empty of meaning.

✗ TypeError

Wrong. Nothing raises here. It runs and quietly gives you `None`, which is what makes the bug hard to see.

The answer is `None`. Methods that change a list in place return `None`: `sort`, `append`, `reverse`, `extend`. The functions that return a NEW value are `sorted()` and `reversed()`.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

### What is printed?

```
months = {"Jan": 1, "Feb": 2}
try:
    print(months[2])
except KeyError as e:
    print("KeyError", e)
```

- a. KeyError 2
- b. 2
- c. Feb
- d. None

✓ **KeyError 2**

Correct. The subscript is a KEY lookup, the key 2 is absent, KeyError is raised and caught, and the missing key is what the exception carries.

✗ **2**

Wrong. Looking up 2 asks for a KEY equal to 2, and there is no such key. The values are 1 and 2, but you cannot look a dictionary up by its value.

✗ **Feb**

Wrong. Nothing here searches the values to find the key that maps to 2. That would need a loop or a reversed dictionary.

✗ **None**

Wrong. A missing key raises rather than returning None. It is `get()` that returns None instead of raising.

The answer is **KeyError 2**. `months[2]` asks for the KEY 2. Use `months.get(2)` to get None instead of a KeyError. This exact confusion is on a past paper.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

- a. [1, 2, 3]
- b. [4]
- c. [1, 2, 3, 4]
- d. [[1, 2, 3], 4]

**X** [1, 2, 3]

Wrong. `b = a` does not copy anything. It gives the same list a second name, so a change through one name is visible through the other.

**X** [4]

Wrong. `append` adds to the existing contents; it does not replace them.

**✓** [1, 2, 3, 4]

Correct. There is only ONE list here with two names, so appending through `b` shows up in `a`.

**X** [[1, 2, 3], 4]

Wrong. `append` adds an item to the list, it does not nest the old list inside a new one.

The answer is [1, 2, 3, 4]. Assignment binds a name, it does not copy. To copy, use `a[:]` or `list(a)` or `a.copy()`. This is the most expensive misunderstanding in the whole course.

The chain below is in the wrong order. What does it print?

```
score = 80
if score >= 40:
    grade = "D"
elif score >= 50:
    grade = "C"
elif score >= 70:
    grade = "A"
print("Grade:", grade)
```

- a. Grade: D
- b. Grade: C
- c. Grade: A
- d. Grade: A then Grade: D

✓ Grade: D

Correct.  $80 \geq 40$  is True at the first test, so the chain stops there and gives D.

✗ Grade: C

Wrong. The chain stopped at the first test.

✗ Grade: A

Wrong. The A test is never reached; the first True test wins.

✗ Grade: A then Grade: D

Wrong. An elif chain runs at most one block.

Order a chain from the most demanding test to the least, or a high score is caught by a low band.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
x = 15
if x > 10:
    print("big")
if x > 5:
    print("medium")
else:
    print("small")
```

- a. big  
medium  
small
- b. medium
- c. big
- d. big  
medium

**X** big  
medium  
small

Wrong. The else belongs to the second if, which was True.

**X** medium

Wrong.  $x > 10$  is also True, so big prints first.

**X** big

Wrong. That would be true with elif; separate ifs are each tested.

**✓** big  
medium

Correct. These are two SEPARATE if statements, so both conditions are tested and both are True.

Separate ifs are all tested, so more than one can run. An elif chain runs at most one.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
age = 16
member = True
if age >= 18:
    if member:
        print("Full")
    else:
        print("Guest")
else:
    print("Too young")
```

- a. Guest
- b. Too young
- c. Full  
Too young
- d. Full

**X** Guest

Wrong. The inner else belongs to the inner if, which is never reached.

**✓** Too young

Correct. `age >= 18` is False, so the outer else runs and the inner if is never reached.

**X** Full

Too young

Wrong. Exactly one outer branch runs.

**X** Full

Wrong. The inner if is only reached when age is at least 18.

An else belongs to the if at the same indentation. The inner decision only happens inside the outer True branch.

3.20

MEDIUM

Lesson 2.1

Which condition correctly catches a score that is NOT between 0 and 100?

- a.  $0 \leq \text{score} \leq 100$
- b.  $\text{score} > 0$  or  $\text{score} < 100$
- c.  $\text{score} < 0$  and  $\text{score} > 100$
- d.  $\text{score} < 0$  or  $\text{score} > 100$

✗  $0 \leq \text{score} \leq 100$

Wrong. That tests for a GOOD score.

✗  $\text{score} > 0$  or  $\text{score} < 100$

Wrong. This is True for almost every number, good or bad.

✗  $\text{score} < 0$  and  $\text{score} > 100$

Wrong. No number is below 0 and above 100 at once, so this is never True.

✓  $\text{score} < 0$  or  $\text{score} > 100$

Correct. A score is bad if it is below 0 OR above 100.

To catch a bad value use or; to confirm a good value use and.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
print(list(range(5, 0, -2)))
```

- a. [5, 3, 1, -1]
- b. [5, 4, 3, 2, 1]
- c. []
- d. [5, 3, 1]

✗ [5, 3, 1, -1]

Wrong. The stop, 0, is excluded, and -1 is past it.

✗ [5, 4, 3, 2, 1]

Wrong. The step is -2, not -1.

✗ []

Wrong. A negative step makes a descending range work.

✓ [5, 3, 1]

Correct. A negative step counts down, stopping before 0.

■ A negative step counts down, and the stop is still excluded.

total is reset **INSIDE** the loop. What does this print?

```
for n in [4, 7, 1]:  
    total = 0  
    total += n  
print(total)
```

- a. 12
- b. 1
- c. an error
- d. 0

✗ 12

Wrong. The reset throws away the earlier additions.

✓ 1

Correct. Each pass sets total back to 0 and adds one number, so only the last, 1, survives.

✗ an error

Wrong. It runs; it just gives the wrong answer.

✗ 0

Wrong. The last pass adds 1 after resetting.

Initialising the total inside the loop resets it every pass, so the answer is always the last number.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
count = 0
for i in range(3):
    for j in range(4):
        count += 1
print(count)
```

- a. 7
- b. 3
- c. 4
- d. 12

✗ 7

Wrong. The counts multiply; they do not add.

✗ 3

Wrong. That counts only the outer passes.

✗ 4

Wrong. That counts only one pass of the inner loop.

✓ 12

Correct. The inner loop runs 4 times for each of the 3 outer passes:  $3 \times 4 = 12$ .

The inner loop runs completely for every pass of the outer one, so multiply the counts.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
xs = [5, 3, 8]
xs.append(1)
xs.insert(1, 9)
print(xs)
```

- a. [5, 9, 3, 8]
- b. [5, 9, 3, 8, 1]
- c. [9, 5, 3, 8, 1]
- d. [5, 3, 8, 1, 9]

✗ [5, 9, 3, 8]

Wrong. append added 1 at the end.

✓ [5, 9, 3, 8, 1]

Correct. append puts 1 at the end, then insert(1, 9) puts 9 at index 1.

✗ [9, 5, 3, 8, 1]

Wrong. insert(1, ...) places the item at index 1, the second position.

✗ [5, 3, 8, 1, 9]

Wrong. insert places 9 at index 1, not at the end.

append(x) adds at the end; insert(i, x) adds at position i.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
xs = [4, 2, 4, 1]
xs.remove(4)
print(xs)
```

- a. [2, 4, 1]
- b. [4, 2, 4]
- c. [4, 2, 1]
- d. [2, 1]

✓ [2, 4, 1]

Correct. remove deletes the FIRST item equal to 4 and leaves the second.

✗ [4, 2, 4]

Wrong. remove takes a value, not an index.

✗ [4, 2, 1]

Wrong. The first 4 is the one removed.

✗ [2, 1]

Wrong. remove deletes only one matching item.

remove(x) deletes the first item equal to x, and raises ValueError if there is none.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
xs = [3, 1, 2]
ys = sorted(xs)
print(xs, ys)
```

- a. [1, 2, 3] [1, 2, 3]
- b. [3, 1, 2] [1, 2, 3]
- c. [1, 2, 3] None
- d. [3, 1, 2] None

✗ [1, 2, 3] [1, 2, 3]

Wrong. sorted() does not change xs.

✓ [3, 1, 2] [1, 2, 3]

Correct. sorted() returns a NEW sorted list and leaves xs alone.

✗ [1, 2, 3] None

Wrong. xs is unchanged and ys holds the sorted copy.

✗ [3, 1, 2] None

Wrong. sorted() returns a list; it is .sort() that returns None.

sorted(xs) gives a new list; xs.sort() sorts in place and returns None.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
xs = [10, 20, 30]
print(xs.pop(), xs)
```

- a. 30 [10, 20, 30]
- b. 10 [20, 30]
- c. 30 [10, 20]
- d. None [10, 20]

✗ 30 [10, 20, 30]

Wrong. pop removes the item as well as returning it.

✗ 10 [20, 30]

Wrong. With no index, pop takes the LAST item.

✓ 30 [10, 20]

Correct. pop() with no argument removes and returns the last item.

✗ None [10, 20]

Wrong. Unlike append and sort, pop returns the item it removed.

pop(i) removes and returns; with no index it takes the last item.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
print([0] * 3 + [1])
```

- a. [3, 1]
- b. [0, 0, 0, 1]
- c. [0, 0, 0, [1]]
- d. [0, 1, 0, 1, 0, 1]

✗ [3, 1]

Wrong. \* repeats the list; it does not multiply the item.

✓ [0, 0, 0, 1]

Correct. [0] \* 3 repeats to [0, 0, 0], and + joins [1] on the end.

✗ [0, 0, 0, [1]]

Wrong. + joins the items of [1], not the list itself.

✗ [0, 1, 0, 1, 0, 1]

Wrong. Only [0] is repeated, before [1] is joined.

| \* on a list repeats it; + joins two lists into one.

**Asked twice.** This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 2.4 and Lesson 3.2 and Lesson 3.4 and Lesson 3.5.

**What happens when this runs?**

```
t = (1, 2, 3)
t[1] = 9
print(t)
```

- a. It raises `TypeError`, because a tuple cannot be changed
- b. It prints (1, 9, 3)
- c. It raises `IndexError`
- d. It prints (1, 2, 3)

✓ It raises `TypeError`, because a tuple cannot be changed

Correct. A tuple is immutable, so item assignment is refused.

✗ It prints (1, 9, 3)

Wrong. The assignment is refused.

✗ It raises `IndexError`

Wrong. Index 1 exists; the problem is changing it.

✗ It prints (1, 2, 3)

Wrong. The error stops the program before the print.

Round brackets are fixed: assigning to a tuple item raises `TypeError`, as with a string.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
print(len(set("hello")))
```

- a. 3
- b. an error, because a string cannot become a set
- c. 5
- d. 4

✗ 3

Wrong. Count the distinct letters: h, e, l, o.

✗ an error, because a string cannot become a set

Wrong. set() accepts any sequence, including a string.

✗ 5

Wrong. Duplicates vanish, so the two l's count once.

✓ 4

Correct. The set of "hello" keeps h, e, l and o, with the repeated l stored once.

A set holds at most one copy of any value, so len() counts distinct items.



**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
d = {"x": 5, "y": 7}
d["x"] = 6
d["z"] = 1
print(len(d), d["x"])
```

a. 3 6

b. 4 6

c. 2 6

d. 3 5

✓ 3 6

Correct. Assigning to an existing key replaces its value; assigning to a new key adds a pair.

✗ 4 6

Wrong. x was replaced, not duplicated; keys are unique.

✗ 2 6

Wrong. z is a new key, so there are three pairs.

✗ 3 5

Wrong. d["x"] = 6 replaced the 5.

**|** d[key] = value adds a new pair or changes an existing one. Keys are unique.



Correct. `items()` walks each pair in insertion order, and only marks of 60 and above print.

Walk the pairs, test the value, print the ones that pass: the shape of every 'display those which' requirement.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
score = 75
if score >= 70:
    print("A")
elif score >= 60:
    print("B")
else:
    print("C")
```

- a. B
- b. A
- B
- c. A
- d. C

✗ B

Wrong. 75 is at or above 60, but the chain never reaches that test: it stopped at the first true one.

✗ A

B

Wrong. An if/elif chain runs AT MOST one block. Separate if statements would print both, which is exactly the trap the paper sets.

✓ A

Correct. The first condition that is true wins, its block runs, and the whole chain is then finished.

✗ C

Wrong. else runs only when every test above it failed.

The answer is A. elif tests only if everything above it failed. Replacing elif with a second if changes the answer, and the paper has asked for exactly that comparison.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
for i in range(1, 6, 2):  
    print(i, end=" ")
```

a. 1 2 3 4 5

b. 1 3 5 7

c. 2 4

d. 1 3 5

✗ 1 2 3 4 5

Wrong. That ignores the third argument, the step of 2.

✗ 1 3 5 7

Wrong. 7 is past the stop value, so it is never produced.

✗ 2 4

Wrong. The start is 1, not 2. range starts where you tell it to.

✓ 1 3 5

Correct. range(start, stop, step) begins at 1, adds 2 each time, and stops BEFORE 6. So 1, 3, 5.

The answer is 1 3 5. range(stop), range(start, stop), range(start, stop, step), and the stop is never included. That last part is the one people lose marks on.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
n = 0
while True:
    n += 1
    if n == 3:
        break
print(n)
```

- a. 3
- b. 0
- c. 2
- d. the loop never ends

✓ 3

Correct. n reaches 3, the if fires, break leaves the loop immediately, and print sees 3.

✗ 0

Wrong. n is incremented before anything else in the body, so it can never still be 0 at the end.

✗ 2

Wrong. break happens AFTER the increment that made n equal to 3, so 3 is the value that survives.

✗ the loop never ends

Wrong. while True does loop forever unless something leaves it, and break is exactly that something.

The answer is 3. break leaves the loop at once; continue jumps to the next iteration. while True with a break inside is the standard way to write a menu loop.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
for i in range(5):  
    if i % 2 == 0:  
        continue  
    print(i, end=" ")
```

a. 0 2 4

b. 1 3

c. 0 1 2 3 4

d. 1 3 5

**X** 0 2 4

Wrong, and it is backwards: continue SKIPS the even numbers, so they are the ones that never print.

**✓** 1 3

Correct. When i is even the continue skips the print, so only the odd values 1 and 3 appear. range(5) stops at 4.

**X** 0 1 2 3 4

Wrong. The continue means not every value reaches the print.

**X** 1 3 5

Wrong. range(5) produces 0 to 4, so 5 is never reached.

The answer is 1 3. continue abandons this pass and starts the next. Reading it as "skip the rest of the body" is the reliable way to trace it.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
for i in range(3):  
    print(i, end=" ")  
else:  
    print("done")
```

- a. 0 1 2
- b. done
- c. 0 1 2 done
- d. 0 1 2 3 done

**X** 0 1 2

Wrong. The else does run, because there is no break in this loop.

**X** done

Wrong. The loop body runs first, three times.

**✓** 0 1 2 done

Correct. A loop else runs when the loop finished normally, that is without a break. Nothing breaks here, so it runs.

**X** 0 1 2 3 done

Wrong. range(3) gives 0, 1 and 2. It never reaches 3.

The answer is 0 1 2 done. A loop else means "no break happened". It is Python-specific and a favourite for a "what is the output" question because it looks like an if else and is not.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
xs = [3, 1, 2]
xs.append(4)
xs.sort()
print(xs)
print(xs[0], xs[-1])
```

a. [3, 1, 2, 4]

3 4

b. [4, 3, 2, 1]

4 1

c. [1, 2, 3]

1 3

d. [1, 2, 3, 4]

1 4

✗ [3, 1, 2, 4]

3 4

Wrong. sort() really does reorder the list. It changes it and returns None.

✗ [4, 3, 2, 1]

4 1

Wrong. sort() ascends by default. Descending needs reverse=True.

✗ [1, 2, 3]

1 3

Wrong. The 4 that append added is still there. append modifies the list itself.

✓ [1, 2, 3, 4]

1 4

Correct. append adds to the end, sort orders the list IN PLACE, and the first and last items are then 1 and 4.

The answer is [1, 2, 3, 4] then 1 4. Lists are ORDERED, so position has meaning, and MUTABLE, so they can be changed in place. The paper asks for both properties by name.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

### What is printed?

```
numbers = [1, 2, 3, 4, 5]
print(numbers[-3:])
print(numbers[:2])
print(numbers[-3:] * 2)
```

a. [2, 3, 4]

[1, 2]

[6, 8, 10]

b. [3, 4, 5]

[1, 2]

[3, 4, 5, 3, 4, 5]

c. [3, 4, 5]

[1, 2, 3]

[6, 8, 10]

d. [3, 4, 5]

[1, 2]

[6, 8, 10]

✗ [2, 3, 4]

[1, 2]

[6, 8, 10]

Wrong. numbers[-3:] counts three back from the end, giving 3, 4 and 5.

✓ [3, 4, 5]

[1, 2]

[3, 4, 5, 3, 4, 5]

Correct. Minus 3 to the end is the last three items. Up to 2 stops before index 2, so two items. A list times 2 REPEATS the list, it does not multiply the numbers in it.

✗ [3, 4, 5]

[1, 2, 3]

[6, 8, 10]

Wrong. `numbers[:2]` excludes index 2, so it is two items, not three.

**X** [3, 4, 5]

[1, 2]

[6, 8, 10]

Wrong on the last line. Star on a list repeats the list; to multiply each number you would need a loop or a comprehension.

The answer is [3, 4, 5] then [1, 2] then [3, 4, 5, 3, 4, 5]. This exact shape, `numbers[-3:] * 2`, is on a past paper. A slice excludes its stop, and star repeats rather than scales.

3.41

HARD

Manual O.3 Q9

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
m = [[1, 2], [3, 4, 5], [6]]  
print(m[1][2])  
print(len(m))
```

a. 4

3

b. 5

6

c. 5

3

d. 3

3

**X** 4

3

Wrong. Index 2 of [3, 4, 5] is 5. Index 1 would be 4.

**X** 5

6

Wrong. len() of a nested list counts the sublists, not the numbers inside them.

**✓** 5

3

Correct. m[1] is the middle list, [3, 4, 5], and its index 2 is 5. len(m) counts the OUTER items, of which there are three.

**X** 3

3

Wrong. m[1][2] reaches into the second list and takes its third item.

The answer is 5 then 3. Read a nested index left to right: first pick the sublist, then index into it.  
The paper prints a three-deep version of this, so do it one bracket at a time on paper.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
s = {3, 1, 2, 3, 1}
print(len(s))
print(sorted(s))
```

a. 5

[1, 1, 2, 3, 3]

b. 3

[1, 2, 3]

c. 3

[3, 1, 2]

d. 5

[1, 2, 3]

✗ 5

[1, 1, 2, 3, 3]

Wrong. A set discards duplicates the moment it is built, so it never held five items.

✓ 3

[1, 2, 3]

Correct. The duplicates collapse, leaving three items, and `sorted()` puts them in order for printing, which is how you show a set reliably.

✗ 3

[3, 1, 2]

Wrong. `sorted()` returns an ordered LIST, so it prints in ascending order whatever order the set happened to be in.

✗ 5

[1, 2, 3]

Wrong. `len()` of that set is 3, not 5, because uniqueness is enforced.

The answer is 3 then [1, 2, 3]. Sets are unique and unordered. That is why the manual compares set answers as sets, never by printed order: Python randomises string hashing per run.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(sorted(a & b))
print(sorted(a | b))
print(sorted(a - b))
```

- a. [1, 2]  
[1, 2, 3, 4, 5]  
[3, 4]
- b. [3, 4]  
[1, 2, 5]  
[1, 2]
- c. [3, 4]  
[1, 2, 3, 4, 5]  
[1, 2]
- d. [3, 4]  
[1, 2, 3, 3, 4, 4, 5]  
[1, 2]

✗ [1, 2]  
[1, 2, 3, 4, 5]  
[3, 4]  
Wrong, and it has intersection and difference swapped.

✗ [3, 4]  
[1, 2, 5]  
[1, 2]  
Wrong. Union is EVERYTHING from both, not only the items unique to one.

✓ [3, 4]  
[1, 2, 3, 4, 5]

[1, 2]

Correct. The ampersand is intersection, what is in both. The bar is union, everything once. The minus is difference, what is in a and not in b.

✗ [3, 4]

[1, 2, 3, 3, 4, 4, 5]

[1, 2]

Wrong. A union is still a set, so nothing appears twice in it.

The answer is [3, 4] then [1, 2, 3, 4, 5] then [1, 2]. Ampersand intersection, bar union, minus difference, caret symmetric difference. The named methods are intersection(), union(), difference() and they mean the same thing.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
d = {"a": 1, "b": 2}
d["c"] = 3
d["a"] = 9
print(d)
print(len(d))
```

a. {'a': 1, 'b': 2, 'c': 3}

3

b. {'a': 9, 'b': 2, 'c': 3}

4

c. {'a': 1, 'a': 9, 'b': 2, 'c': 3}

4

d. {'a': 9, 'b': 2, 'c': 3}

3

✗ {'a': 1, 'b': 2, 'c': 3}

3

Wrong. `d["a"] = 9` really does overwrite the 1.

✗ {'a': 9, 'b': 2, 'c': 3}

4

Wrong. There are three keys, not four. Overwriting a key does not add one.

✗ {'a': 1, 'a': 9, 'b': 2, 'c': 3}

4

Wrong. A dictionary cannot hold the same key twice. That is what makes it a mapping.

✓ {'a': 9, 'b': 2, 'c': 3}

3

Correct. Assigning to a key that does not exist ADDS it; assigning to one that does REPLACES its value. Three keys remain.

The answer is {'a': 9, 'b': 2, 'c': 3} then 3. Keys are unique; assigning to an existing key replaces its value. Since Python 3.7 a dictionary also keeps insertion order, which is why the printed order is predictable.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
d = {"x": 1, "y": 2}
for k in d:
    print(k, end=" ")
print()
for k, v in d.items():
    print(k, v, end=" ")
```

a. 1 2

x 1 y 2

b. x y

x 1 y 2

c. ('x', 1) ('y', 2)

x 1 y 2

d. x y

1 2

**X** 1 2

x 1 y 2

Wrong. The plain loop gives keys, not values. For values you would write `d.values()`.

**✓** x y

x 1 y 2

Correct. Iterating a dictionary directly gives its KEYS. `items()` gives key and value pairs, which unpack into the two loop variables.

**X** ('x', 1) ('y', 2)

x 1 y 2

Wrong. The plain loop yields keys alone, not pairs. Pairs come from `items()`.

**X** x y

1 2

Wrong on the second line: `items()` gives both parts, and both are printed.

The answer is `x y` then `x 1 y 2`. `for k in d` gives keys. `d.keys()`, `d.values()`, `d.items()` give what they say. Forgetting `items()` and then trying to unpack a key is a common exam slip.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
print("a" in "cat")
print(2 in [1, 2, 3])
print("x" in {"x": 1})
```

- a. True  
True  
False
- b. False  
True  
True
- c. True  
False  
True
- d. True  
True  
True

**X** True  
True  
False

Wrong on the third line. in on a dictionary searches the keys, and "x" is a key here.

**X** False  
True  
True

Wrong on the first line. "a" is one of the characters of "cat", so it is found.

**X** True  
False  
True

Wrong on the second line. 2 is an item of that list.

✓ True

True

True

Correct. On a string, in tests for a substring. On a list, for an item. On a dictionary, for a KEY.

The answer is True then True then True. in means substring on a string, membership on a list or set, and KEY membership on a dictionary. That last one is the trap.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
xs = [10, 20, 30, 40]
del xs[1]
print(xs)
d = {"a": 1, "b": 2}
del d["a"]
print(d)
```

a. [10, 30, 40]

{'b': 2}

b. [20, 30, 40]

{'b': 2}

c. [10, 20, 30]

{'a': 1}

d. [10, 30, 40]

{'a': 1}

✓ [10, 30, 40]

{'b': 2}

Correct. del removes by POSITION on a list, so index 1 (the 20) goes, and by KEY on a dictionary.

✗ [20, 30, 40]

{'b': 2}

Wrong. Index 1 is the second item. Index 0 is the 10.

✗ [10, 20, 30]

{'a': 1}

Wrong. del xs[1] takes out the second item, not the last, and del d["a"] takes out the "a" entry.

✗ [10, 30, 40]

{'a': 1}

Wrong on the dictionary: del d["a"] removes the key it names.

The answer is [10, 30, 40] then {'b': 2}. del removes by index on a list and by key on a dictionary. remove() removes by VALUE on a list, and pop() removes and gives the item back.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
for n in range(1, 8):  
    if n == 5:  
        break  
    if n % 2 == 0:  
        continue  
    print(n)
```

a. 1

3

5

7

b. 1

2

3

4

c. 1

3

d. 1

3

5

**X** 1

3

5

7

Wrong. break stops the loop at 5.

✗ 1

2

3

4

Wrong. continue skips the even numbers.

✓ 1

3

Correct. Even numbers are skipped by continue, and at 5 break ends the loop before printing.

✗ 1

3

5

Wrong. break runs before 5 can be printed.

break leaves the loop at once; continue skips the rest of this pass.

The user types 5, then 8, then 0, then 3. The list stands in for input(). What does this print?

```
typed = [5, 8, 0, 3]    # stands in for four input() calls
i = 0
total = 0
count = 0
number = typed[i]
while number != 0:
    total += number
    count += 1
    i += 1
    number = typed[i]
print(total, count)
```

- a. 13 2
- b. 16 4
- c. 13 3
- d. 16 3

✓ 13 2

Correct. 5 and 8 are added; the 0 stops the loop before it is added or counted, and the 3 is never read.

✗ 16 4

Wrong. Neither the 0 nor the 3 is counted.

✗ 13 3

Wrong. The 0 is not counted, because the body never runs for it.

✗ 16 3

Wrong. The loop stops at 0, so 3 is never added.

A sentinel loop reads once before the loop and again at the bottom. The sentinel itself is never added or counted.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
a = [1, 2]
a.append([3, 4])
b = [1, 2]
b += [3, 4]
print(len(a), len(b))
```

a. 3 3

b. 4 4

c. 4 3

d. 3 4

✗ 3 3

Wrong. += extends the list item by item.

✗ 4 4

Wrong. append adds the whole list as a single item.

✗ 4 3

Wrong. The two results are the other way round.

✓ 3 4

Correct. append adds [3, 4] as ONE item, giving 3; += adds 3 and 4 as separate items, giving 4.

| append adds one item; += and extend add the items of the other list one by one.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
n = [[1, [2, 3]], [4, 5]]  
print(n[0][1][0], len(n[0]))
```

- a. 2 2
- b. 1 2
- c. 2 3
- d. [2, 3] 2

✓ 2 2

Correct. `n[0]` is `[1, [2, 3]]`; `[1]` of that is `[2, 3]`; `[0]` of that is 2. `n[0]` has two items.

✗ 1 2

Wrong. Peel one bracket at a time: `n[0]`, then `[1]`, then `[0]`.

✗ 2 3

Wrong. `len` counts items: `n[0]` holds a number and a list, two items.

✗ [2, 3] 2

Wrong. The last `[0]` reaches inside `[2, 3]`.

Peel a nested index one bracket at a time and write down what you have after each; `len` counts items, not the numbers inside them.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
t = (5,)
u = (5)
print(type(t).__name__, type(u).__name__)
```

- a. tuple tuple
- b. int int
- c. tuple int
- d. int tuple

✗ tuple tuple

Wrong. Without a comma the brackets are only grouping.

✗ int int

Wrong. The comma makes t a tuple.

✓ tuple int

Correct. (5,) is a tuple because of the comma; (5) is just the number 5 in brackets.

✗ int tuple

Wrong. They are the other way round.

! A tuple of one item needs a comma: (5,) is a tuple, (5) is 5.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
a = [1, 2]
b = list(a)
b.append(3)
print(a, b)
```

- a. [1, 2, 3] [1, 2]
- b. [1, 2] [1, 2]
- c. [1, 2, 3] [1, 2, 3]
- d. [1, 2] [1, 2, 3]

✗ [1, 2, 3] [1, 2]

Wrong. a is the one left unchanged.

✗ [1, 2] [1, 2]

Wrong. b did get the 3.

✗ [1, 2, 3] [1, 2, 3]

Wrong. That is what `b = a` would do; `list(a)` is a copy.

✓ [1, 2] [1, 2, 3]

Correct. `list(a)` makes a NEW list, so appending to b leaves a alone.

Assignment does not copy: `b = a` gives one list two names. `list(a)` or `a[:]` makes a real copy.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
print(type({}).__name__, type(set().__name__))
```

- a. set set
- b. dict dict
- c. dict set
- d. set dict

✗ set set

Wrong. {} is a dictionary.

✗ dict dict

Wrong. set() makes a set.

✓ dict set

Correct. {} on its own is an empty **DICTIONARY**; an empty set must be written set().

✗ set dict

Wrong. They are the other way round.

| {} alone is an empty dictionary, not an empty set.

**Asked twice.** This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 3.2 and Lesson 3.4 and Lesson 3.5.

**What happens when this runs?**

```
d = {[1, 2]: "a"}  
print(d)
```

- a. It raises `TypeError`, because a list cannot be a dictionary key
- b. It raises `KeyError`
- c. It prints `{[1, 2]: 'a'}`
- d. It prints `a`

✓ It raises `TypeError`, because a list cannot be a dictionary key

Correct. A key must be hashable, which means immutable; a list is mutable.

✗ It raises `KeyError`

Wrong. Nothing is looked up; the key itself is rejected.

✗ It prints `{[1, 2]: 'a'}`

Wrong. The dictionary cannot be built.

✗ It prints `a`

Wrong. The error happens when the dictionary is made.

Strings, numbers and tuples can be keys; lists and sets cannot, because they can change.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
d = {"x": 1}
print(1 in d, "x" in d)
```

- a. True True
- b. False True
- c. True False
- d. False False

**X True True**

Wrong. 1 is a value, and in does not look at values.

**✓ False True**

Correct. in on a dictionary checks the KEYS. 1 is a value, not a key; "x" is a key.

**X True False**

Wrong. They are the other way round.

**X False False**

Wrong. "x" is a key, so that test is True.

**|** in means a substring on a string, an item on a list or set, and a KEY on a dictionary.

## 4 Functions, Modules, Files and Exceptions

4.1

EASY

Manual O.4 Q6

A file is opened with `open("data.txt", "w")` when `data.txt` already exists and contains text. What happens?

- a. The new text is added after the existing text
- b. Python raises `FileExistsError`
- c. The existing contents are erased
- d. The file is opened read-only

✗ The new text is added after the existing text

Wrong, and this is the mode people mean when they choose "w" by mistake. Appending is "a".

✗ Python raises `FileExistsError`

Wrong. "w" is perfectly happy for the file to exist. It just empties it. The mode that refuses to overwrite is "x".

✓ The existing contents are erased

Correct. "w" truncates the file to nothing the moment it is opened, before you write a single character. Data is lost even if the program then crashes.

✗ The file is opened read-only

Wrong. "w" is write-only. Reading is "r", which is also the default.

The answer is The existing contents are erased. "r" read, "w" write and TRUNCATE, "a" append, "x" create only if absent. Choosing "w" where you meant "a" destroys the file, and the paper asks about this.

A text file holds three lines. Which call gives a LIST of those three lines?

- a. `f.readlines()`
- b. `f.read()`
- c. `f.readline()`
- d. `f.line()`

✓ `f.readlines()`

Correct. `readlines()` gives a list with one string per line, each still carrying its trailing newline unless you strip it.

✗ `f.read()`

Wrong. `read()` gives the WHOLE file as one string, newlines and all.

✗ `f.readline()`

Wrong. `readline()` gives ONE line, the next one, as a string.

✗ `f.line()`

Wrong. There is no such method.

The answer is `f.readlines()`. read whole, readline one, readlines list. Note the newline stays on each item, so `print()` of one adds a blank line unless you strip.

### What is a docstring?

- a. A comment starting with a hash inside a function
- b. A special variable holding the function name
- c. A string literal as the first statement of a function, module or class, describing what it does
- d. A file that documents the whole program

**X** A comment starting with a hash inside a function

Wrong. A hash comment is stripped and cannot be read back at run time. A docstring survives into the object.

**X** A special variable holding the function name

Wrong. That is `__name__`, a different attribute entirely.

**✓** A string literal as the first statement of a function, module or class, describing what it does

Correct. It sits directly under the `def` line, usually in triple quotes, and becomes the object's `__doc__`, which is what `help()` shows you.

**X** A file that documents the whole program

Wrong. A docstring lives inside the code, not beside it.

The answer is A string literal as the first statement of a function, module or class, describing what it does. First statement, a string, becomes `__doc__`. It is the "self-documenting" part of the modularisation answer.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
def area(l, w):  
    return l * w  
  
print(area(3, 4) + area(1, 2))
```

- a. 14
- b. None
- c. 12
- d. 7

✓ 14

Correct. `area(3, 4)` returns 12 and `area(1, 2)` returns 2;  $12 + 2$  is 14.

✗ None

Wrong. The function returns its result, so the calls can be added.

✗ 12

Wrong. The second call adds 2 more.

✗ 7

Wrong. The function multiplies, it does not add.

Because a value returning function hands back its result, a call can be used anywhere a value can.

[illegible]

### What does this print?

```
def hello():
    print("hello")

print("bye")
```

- hello
- hello  
bye
- nothing at all
- bye

X hello

Wrong. `print("bye")` runs; the function body does not.

**X** hello

bye

Wrong. A definition on its own produces no output.

**X** nothing at all

Wrong. The last line does print.

✓ bye

Correct. `hello` is defined but never called, so its body never runs.

Nothing inside a function happens until it is called.

4.6

EASY

Lesson 3.1

In `def cube(num):` and the call `cube(4)`, which is the ARGUMENT?

- a. return, the keyword
- b. cube, the function's name
- c. num, the name in the definition
- d. 4, the value supplied at the call

✗ return, the keyword

Wrong. return hands a value back.

✗ cube, the function's name

Wrong. That is the function's name.

✗ num, the name in the definition

Wrong. num is the PARAMETER.

✓ 4, the value supplied at the call

Correct. The argument is the value passed in when you call.

| The parameter is the name in the definition; the argument is the value supplied at the call.

4.7

EASY

Lesson 3.1

True or false: a function with no return statement gives back None.

Answer: **true**

| Show or give: print shows, return hands back, and no return means None.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
import math
print(math.ceil(2.1), math.floor(2.9))
```

a. 3 2

b. 2 2

c. 3 3

d. 2 3

✓ 3 2

Correct. ceil always rounds UP to the next whole number and floor always rounds DOWN.

✗ 2 2

Wrong. ceil(2.1) goes up to 3.

✗ 3 3

Wrong. floor(2.9) goes down to 2.

✗ 2 3

Wrong. They are the other way round.

math.ceil rounds up, math.floor rounds down, round goes to the nearest.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
from math import sqrt  
print(sqrt(49))
```

- a. an error, because sqrt needs the math. prefix
- b. 49
- c. 7.0
- d. 7

**X** an error, because sqrt needs the math. prefix

Wrong. The from form brings the name in, so no prefix is needed.

**X** 49

Wrong. sqrt is the square root, not the square.

**✓** 7.0

Correct. from math import sqrt brings sqrt in directly, and sqrt always returns a float.

**X** 7

Wrong. sqrt returns a float, 7.0.

from math import sqrt gives you sqrt alone; import math gives you math.sqrt.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
import math
print(round(math.pi, 3))
```

- a. 3
- b. 3.1416
- c. 3.142
- d. 3.14

**X** 3

Wrong. round with 3 keeps three decimal places.

**X** 3.1416

Wrong. That is four decimal places.

**✓** 3.142

Correct. round(math.pi, 3) keeps three decimal places.

**X** 3.14

Wrong. That is two decimal places.

**|** When the paper says use the math module, use math.pi, rounded as asked.

4.11

EASY

Lesson 3.2

What is a module, in the course's words?

- a. A single function inside a program
- b. A file of Python definitions and statements that other programs can import and reuse
- c. A folder of databases
- d. A window in a graphical interface

✗ A single function inside a program

Wrong. A module is a whole file, which may hold many functions.

✓ A file of Python definitions and statements that other programs can import and reuse

Correct. That is the definition.

✗ A folder of databases

Wrong. A directory of MODULES is a package.

✗ A window in a graphical interface

Wrong. That is a tkinter window.

| A module is a file of Python code that can be imported; a package is a directory of modules.

4.12

EASY

Lesson 3.2

True or false: after `import math` you can write `sqrt(16)` without the `math.` prefix.

Answer: **false**

| `import math` gives `math` dot something; `from math import sqrt` gives `sqrt` alone.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
with open("t.txt", "w") as f:
    f.write("one\n")
with open("t.txt", "a") as f:
    f.write("two\n")
print(open("t.txt").read().strip())
```

a. one

two

b. two

c. one

d. two

one

✓ one

two

Correct. "a" opens at the end of the file, so the new line goes after the old one.

✗ two

Wrong. Append keeps the existing content.

✗ one

Wrong. The appended line is there too.

✗ two

one

Wrong. Appended data goes at the end.

Append adds new data at the end and leaves everything already there.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
with open("t.txt", "w") as f:  
    f.write("x\ny\nz\n")  
print(len(open("t.txt").readlines()))
```

- a. 1
- b. 6
- c. 5
- d. 3

✗ 1

Wrong. That is how many strings `read()` returns.

✗ 6

Wrong. `len` counts lines here, not characters.

✗ 5

Wrong. There are three lines.

✓ 3

Correct. `readlines()` returns a list with one string per line: `x`, `y` and `z`.

| `read()` gives one string, `readline()` one line, `readlines()` a list of every line.

**Asked twice.** This same question also appears as: Manual O.4 Q10.

**Which exception does this raise?**

```
print("5" + 5)
```

- a. NameError
- b. ValueError
- c. SyntaxError
- d. TypeError

**X** NameError

Wrong. Nothing here is undefined.

**X** ValueError

Wrong. ValueError is the right kind with an impossible value, like `int("abc")`.

**X** SyntaxError

Wrong. The line is legal Python; it fails only when run.

**✓** TypeError

Correct. `+` cannot join a string and a number: the wrong KIND of thing.

| TypeError is the wrong kind; ValueError is the right kind with an impossible value.

Which of these is **NOT** a benefit of breaking a program into functions?

- a. The program runs measurably faster because it is shorter
- b. The same code can be reused instead of repeated
- c. Each piece can be tested on its own
- d. The program is easier to read and to maintain

✓ The program runs measurably faster because it is shorter

Correct. Calling a function costs a little time rather than saving it. The benefits of modularising are all about the PROGRAMMER: reuse, testing, readability, maintainability, teamwork. Speed is not among them.

✗ The same code can be reused instead of repeated

Wrong choice, because reuse IS a benefit, and usually the first one listed.

✗ Each piece can be tested on its own

Wrong choice, because testing a piece in isolation IS a benefit.

✗ The program is easier to read and to maintain

Wrong choice, because readability and maintenance ARE benefits, and the paper lists them.

The answer is The program runs measurably faster because it is shorter. Every benefit of modularising is a benefit to the person reading or changing the code. Speed is not one, and a question offering it is offering the distractor.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
import math
from math import sqrt
print(math.sqrt(16), sqrt(16))
```

- a. 4 4
- b. 4.0 4.0
- c. 4.0 4
- d. NameError

✗ 4 4

Wrong. `math.sqrt` returns a float even for a perfect square, so the point and zero are printed.

✓ 4.0 4.0

Correct. Both forms reach the same function. `sqrt` always returns a float, so both print 4.0.

✗ 4.0 4

Wrong. Both calls are the same function and give the same value.

✗ NameError

Wrong. Both imports are valid, and each makes its name available in its own way.

The answer is 4.0 4.0. `import math` needs the `math.` prefix; `from math import sqrt` puts the name in directly. Both are examinable and the difference is which name you must then write.

Why is `with open(...)` as `f` preferred to `f = open(...)`?

- a. It is faster to type
- b. It is the only way to read a file line by line
- c. It opens the file in a special faster mode
- d. It closes the file automatically, even if an error is raised inside the block

✗ It is faster to type

Wrong. Brevity is not the reason, and it is barely shorter anyway.

✗ It is the only way to read a file line by line

Wrong. A plain `open()` can be iterated line by line just as well. The difference is only about closing.

✗ It opens the file in a special faster mode

Wrong. The mode is whatever you pass to `open`. `with` changes nothing about how the file is opened.

✓ It closes the file automatically, even if an error is raised inside the block

Correct. The `with` block guarantees `f.close()` runs on the way out, whether the block finished normally or raised. Without it, an exception can leave the file open and its buffer unwritten.

The answer is It closes the file automatically, even if an error is raised inside the block. `with` guarantees the close. It is the same guarantee finally gives, written more briefly, and saying that in the exam earns the mark.

**Asked twice.** This same question also appears as: Lesson 3.4.

**Which exception does this raise?**

```
xs = [1, 2, 3]
print(xs[5])
```

- a. KeyError
- b. ValueError
- c. TypeError
- d. IndexError

**X** KeyError

Wrong. KeyError is for a missing DICTIONARY key. A list is indexed by position, not by key.

**X** ValueError

Wrong. ValueError is for a value of the right type that cannot be used, such as `int("abc")`.

**X** TypeError

Wrong. TypeError would be `xs["a"]`, where the index is the wrong KIND of thing entirely.

**✓** IndexError

Correct. The index is outside the range of the list, which is exactly what IndexError means.

The answer is IndexError. IndexError for a sequence, KeyError for a mapping. Naming the right exception is worth marks on its own in this paper.

**Why is `except:` on its own considered bad practice?**

- a. It catches everything, including errors you did not anticipate, and hides real bugs
- b. It is slower than naming the exception
- c. It is not valid Python
- d. It can only be used once per program

✓ It catches everything, including errors you did not anticipate, and hides real bugs

Correct. A bare `except` swallows every exception, including a typo raising `NameError`, so the program keeps going in a broken state and the real fault is invisible.

✗ It is slower than naming the exception

Wrong. Speed is not the issue and the difference is immeasurable.

✗ It is not valid Python

Wrong. It is valid, which is exactly why it is dangerous.

✗ It can only be used once per program

Wrong. There is no such limit.

The answer is It catches everything, including errors you did not anticipate, and hides real bugs. Catch the exception you expect and no more. A bare `except` turns a loud failure into a silent wrong answer.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q14.

### What is printed?

```
def check(age):  
    if age < 0:  
        raise ValueError("negative age")  
    return age  
  
try:  
    check(-1)  
except ValueError as e:  
    print(e)
```

- a. ValueError
- b. negative age
- c. -1
- d. nothing is printed

#### ValueError

Wrong. Printing the exception object shows its MESSAGE, not the name of its class. For the class name you would print `type(e).__name__`.

#### negative age

Correct. `raise` creates the exception with that message, the `except` catches it and binds it to `e`, and printing an exception prints its message.

#### -1

Wrong. The argument is never returned, because the `raise` happens first.

#### nothing is printed

Wrong. The `except` block does run and does print.

The answer is negative age. `raise` makes your own error, and validating input this way is what the paper means by defensive programming.



**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
def power(base, exp=2):  
    return base ** exp  
  
print(power(3), power(2, 3), power(exp=1, base=9))
```

a. 6 8 9

b. 9 8 1

c. 9 6 9

d. 9 8 9

✗ 6 8 9

Wrong. `**` is power, so `3 ** 2` is 9.

✗ 9 8 1

Wrong. `exp=1, base=9` is 9 to the power 1, which is 9.

✗ 9 6 9

Wrong. `2 ** 3` is 8.

✓ 9 8 9

Correct. `power(3)` uses the default exponent 2; `power(2, 3)` is 8; named arguments give `9 ** 1`.

A parameter with a default can be left out; arguments passed by name can come in any order.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
def stats(xs):  
    return min(xs), max(xs)  
  
lo, hi = stats([4, 9, 2])  
print(hi - lo)
```

- a. 11
- b. 2 9
- c. (2, 9)
- d. 7

**X** 11

Wrong. The program subtracts, it does not add.

**X** 2 9

Wrong. The print shows hi - lo, one number.

**X** (2, 9)

Wrong. The two values were unpacked into separate names.

**✓** 7

Correct. stats returns 2 and 9, which unpack into lo and hi; 9 - 2 is 7.

**|** return a, b hands back two values, unpacked with two names on the left.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
def f():  
    return 1  
    print("after")  
  
print(f())
```

a. 1  
after

b. None

c. after  
1

d. 1

✗ 1  
after

Wrong. Nothing after return runs.

✗ None

Wrong. The function returns 1.

✗ after  
1

Wrong. The line after return is never reached.

✓ 1

Correct. return hands back 1 and leaves the function at once, so the print after it never runs.

| return hands a value back AND leaves the function.

Which two of these are benefits of modularising a program with functions?

- a. Easier maintenance: a change made in one place takes effect everywhere
- b. The program always runs faster
- c. Reusability: write once, call from many places
- d. Every variable becomes global

✓ Easier maintenance: a change made in one place takes effect everywhere

Correct. Easier maintenance is one of the five benefits.

✗ The program always runs faster

Wrong. The course says modularising does NOT make the program run faster.

✓ Reusability: write once, call from many places

Correct. Reusability is one of the five benefits.

✗ Every variable becomes global

Wrong. Names made inside a function are local.

Five benefits: reusability, readability, easier debugging and testing, easier maintenance, teamwork and simpler design.

**Asked twice.** This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.4 and Lesson 3.5.

**What happens when this runs?**

```
from math import pi  
print(math.pi)
```

- a. It raises `NameError`, because the name `math` was never imported
- b. It raises `ImportError`
- c. It prints `3.14`
- d. It prints `3.141592653589793`

✓ It raises `NameError`, because the name `math` was never imported  
Correct. `from math import pi` brings in `pi` alone; `math` itself is not a name in the program.

✗ It raises `ImportError`  
Wrong. The import itself succeeded.

✗ It prints `3.14`  
Wrong. The line fails before printing anything.

✗ It prints `3.141592653589793`  
Wrong. `math.pi` needs `import math`.

! The two forms are not interchangeable: after `from math import pi`, write `pi`, not `math.pi`.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
import math as m
print(m.sqrt(9))
```

- a. 9
- b. 3
- c. an error, because math was renamed
- d. 3.0

**X** 9

Wrong. sqrt is the square root.

**X** 3

Wrong. sqrt returns a float.

**X** an error, because math was renamed

Wrong. Renaming is allowed; you then use the new name.

**✓** 3.0

Correct. import math as m renames the module, so m.sqrt(9) is 3.0.

**|** import math as m gives the module a shorter name; everything is then reached through m.

What can `random.randint(1, 6)` return?

- a. Always 1
- b. A decimal between 1 and 6
- c. A whole number from 1 to 6, with both 1 and 6 possible
- d. A whole number from 1 to 5

✗ Always 1

Wrong. It is random.

✗ A decimal between 1 and 6

Wrong. `randint` gives whole numbers.

✓ A whole number from 1 to 6, with both 1 and 6 possible

Correct. `randint(a, b)` includes both ends.

✗ A whole number from 1 to 5

Wrong. Unlike `range`, `randint` includes its upper limit.

`random.randint(a, b)` gives a whole number from `a` to `b` inclusive; `random.choice(seq)` picks one item.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
with open("t.txt", "w") as f:
    f.write("one\n")
with open("t.txt", "w") as f:
    f.write("two\n")
print(open("t.txt").read().strip())
```

- a. an error, because the file already exists
- b. two
- c. one
- d. one  
two

**X** an error, because the file already exists

Wrong. "w" happily opens an existing file, and empties it.

**✓** two

Correct. Opening with "w" a second time empties the file, so only the second line is left.

**X** one

Wrong. The second "w" replaced the first line.

**X** one

two

Wrong. That is what append mode, "a", would give.

**|** w wipes, a adds, r reads.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
with open("t.txt", "w") as f:  
    f.write("a")  
    f.write("b")  
print(open("t.txt").read())
```

- a. a  
b
- b. b
- c. ab
- d. a b

**X** a  
b

Wrong. write() does not add a newline for you.

**X** b

Wrong. Both writes happen inside the same open file.

**✓** ab

Correct. write() puts exactly what it is given; with no `\n` the two pieces join.

**X** a b

Wrong. No space was written.

**|** Always write the newline yourself: without `\n` every record runs into the next.

No file called gone.txt exists. What happens when this runs?

```
f = open("gone.txt", "r")  
print(f.read())
```

- a. It raises FileNotFoundError
- b. It creates an empty gone.txt
- c. It raises KeyError
- d. It prints an empty line

✓ It raises FileNotFoundError

Correct. Read mode needs the file to exist.

✗ It creates an empty gone.txt

Wrong. Only "w" and "a" create a missing file.

✗ It raises KeyError

Wrong. KeyError is for dictionaries.

✗ It prints an empty line

Wrong. The open fails before any read.

! "r" raises FileNotFoundError if the file is missing; "w" and "a" create it.

The file holds seven records. What does this print?

```
with open("r.txt", "w") as f:
    for i in range(1, 8):
        f.write(f"Student {i}\n")
with open("r.txt") as f:
    lines = f.readlines()
for line in lines[:2]:
    print(line.strip())
```

- a. Student 1  
Student 2
- b. Student 1  
Student 2  
Student 3
- c. Student 1
- d. Student 7

✓ Student 1  
Student 2

Correct. `lines[:2]` takes the first two lines, and `strip()` removes each newline.

✗ Student 1  
Student 2  
Student 3

Wrong. A slice stops before its stop index, 2.

✗ Student 1

Wrong. Two lines are taken.

✗ Student 7

Wrong. `[:2]` takes from the start.

Given a list of lines, `lines[:5]` is the first five: a one-line answer to the paper's requirement.

Which mode opens a file for BOTH reading and writing, at the start, without truncating it?

- a. "a"
- b. "w"
- c. "r"
- d. "r+"

✗ "a"

Wrong. a writes at the end.

✗ "w"

Wrong. w erases the file.

✗ "r"

Wrong. r reads only.

✓ "r+"

Correct. r+ reads and writes, keeps the content, and raises if the file is missing.

The four modes the paper asks for: r read, w write and truncate, a append, r+ read and write.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
try:
    print("a")
    x = int("b")
    print("c")
except ValueError:
    print("d")
print("e")
```

a. a

d

e

b. a

d

c. d

e

d. a

c

d

e

✓ a

d

e

Correct. a prints, int("b") raises ValueError, the rest of the try is skipped, except prints d, and the program carries on to e.

X a  
d

Wrong. After a handled exception the program continues.

X d  
e

Wrong. a printed before the error.

X a  
c  
d  
e

Wrong. c is skipped: the error jumps straight to except.

When a line in try raises, the rest of the try is skipped and the matching except runs; then the program continues.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
try:
    n = 10 // 2
except ZeroDivisionError:
    print("zero")
else:
    print("ok", n)
finally:
    print("end")
```

- a. zero  
end
- b. ok 5  
end
- c. end
- d. ok 5

**X** zero  
end

Wrong. `10 // 2` does not divide by zero.

**✓** ok 5  
end

Correct. Nothing is raised, so `else` runs; `finally` runs every time.

**X** end

Wrong. `else` runs because the `try` succeeded.

**X** ok 5

Wrong. `finally` always runs.

else runs when nothing was raised; finally runs either way.

4.37

MEDIUM

Lesson 3.4

Now the division is by zero. What does this print?

```
try:
    n = 10 // 0
except ZeroDivisionError:
    print("zero")
else:
    print("ok")
finally:
    print("end")
```

- a. ok  
end
- b. end
- c. zero  
end
- d. zero

**X** ok  
end

Wrong. else runs only when nothing was raised.

**X** end

Wrong. The except block prints first.

**✓** zero  
end

Correct. The except runs, else is skipped because the try failed, and finally still runs.

**X** zero

Wrong. finally always runs.

Finally means finally: it runs whether the try worked or not.

### Can try and except catch a missing colon in the same file?

- a. Yes, with a bare except
- b. Yes, with except `SyntaxError`
- c. Only inside a function
- d. No: a syntax error stops the program from running at all, so no try block ever runs

✗ Yes, with a bare except

Wrong. Nothing runs at all.

✗ Yes, with except `SyntaxError`

Wrong. The try block never gets the chance to run.

✗ Only inside a function

Wrong. A syntax error anywhere stops the whole file.

✓ No: a syntax error stops the program from running at all, so no try block ever runs

Correct. Python cannot parse the file, so nothing runs, the try included.

| Syntax errors stop it starting. Exceptions stop it finishing, and only those can be caught.

Which two of these are exceptions raised while a program RUNS, rather than syntax errors?

- a. ZeroDivisionError, from `10 / 0`
- b. A string that opens with `"` and closes with `'`
- c. KeyError, from `d["missing"]`
- d. A missing colon after an `if`

✓ ZeroDivisionError, from `10 / 0`

Correct. `10 / 0` is legal code that fails only when it runs.

✗ A string that opens with `"` and closes with `'`

Wrong. That is a syntax error too.

✓ KeyError, from `d["missing"]`

Correct. A missing key is found only when the lookup runs.

✗ A missing colon after an `if`

Wrong. That is a syntax error, found before anything runs.

■ An exception is an error detected during execution; a syntax error is bad grammar found first.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
def add(a, b=10):  
    return a + b  
  
print(add(5))  
print(add(5, 1))  
print(add(b=2, a=3))
```

a. 15

6

6

b. 15

6

5

c. 5

6

5

d. TypeError

6

5

✗ 15

6

6

Wrong on the last line:  $3 + 2$  is 5, not 6.

✓ 15

6

5

Correct. The default fills in for  $b$  when only one argument is given. Positional arguments fill left to right. Keyword arguments may be given in any order because they name their parameter.

X 5

6

5

Wrong on the first line. b defaults to 10, so add(5) is 15.

X TypeError

6

5

Wrong. add(5) is legal precisely BECAUSE b has a default.

The answer is 15 then 6 then 5. A parameter with a default may be omitted. Keyword arguments name their target, so order stops mattering. Defaults must come after non-defaults in the definition.

4.41

HARD

Manual O.4 Q2

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
def f(x):  
    print(x * 2)  
  
def g(x):  
    return x * 2  
  
a = f(3)  
b = g(3)  
print(a, b)
```

- a. 6  
6 6
- b. 6 6  
None 6
- c. 6  
None 6
- d. None 6

**X** 6  
6 6

Wrong. a is None. A function with no return statement gives back None.

**X** 6 6  
None 6

Wrong. f prints once, when it is called; the value it gives back is still None.

**✓** 6  
None 6

Correct. f prints 6 and returns nothing, so a is None. g returns 6 silently, so b is 6. The last line then prints None 6.

**X** None 6

Wrong. The 6 printed inside f appears first, on its own line.

The answer is 6 then None 6. print SHOWS a value, return HANDS IT BACK. A function that only prints returns None, and using that None later is a standard exam bug.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
x = 1

def f():
    x = 2
    print(x)

f()
print(x)
```

a. 2

2

b. 1

1

c. 1

2

d. 2

1

X 2

2

Wrong. Assigning inside a function does not change the global unless the function declares global x.

X 1

1

Wrong. Inside f, the local x shadows the global one, so 2 prints first.

X 1

2

Wrong, and it has the two the wrong way round.

✓ 2

1

Correct. The assignment inside `f` creates a LOCAL `x`. The global `x` is untouched, so it is still 1 afterwards.

The answer is 2 then 1. Assigning to a name inside a function makes it local. To change a global you must say `global x`, and this pair of prints is exactly how the paper tests it.

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q12 and Manual O.4 Q14.

**What is printed?**

```
try:
    x = 1 / 0
except ZeroDivisionError:
    print("caught")
finally:
    print("always")
```

- a. always
- b. caught  
always
- c. caught
- d. the program crashes

**X** always

Wrong. The except block ran too, and its print came first.

**✓** caught  
always

Correct. The division raises, the matching except handles it, and finally runs afterwards as it always does.

**X** caught

Wrong. finally runs whether or not anything was raised. That is its whole point.

**X** the program crashes

Wrong. The exception was caught, so nothing propagates and nothing crashes.

The answer is caught then always. try, except by exception type, else if nothing was raised, finally always. finally is where you put the cleanup that must happen either way.

4.44

HARD

Manual O.4 Q14

**Asked twice.** This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12.

**What is printed?**

```
def fact(n):  
    if n <= 1:  
        return 1  
    return n * fact(n - 1)  
  
print(fact(5))
```

- a. 15
- b. 5
- c. RecursionError
- d. 120

**X** 15

Wrong. 15 is  $1+2+3+4+5$ , the SUM. This function multiplies.

**X** 5

Wrong. The function does not simply return its argument; it multiplies down to the base case.

**X** RecursionError

Wrong. There IS a base case, so the recursion terminates. Without one, RecursionError would be right.

**✓** 120

Correct.  $5 * 4 * 3 * 2 * 1$  is 120. The base case at  $n \leq 1$  stops the recursion.

The answer is 120. Every recursion needs a base case and a step that moves towards it. Missing the base case gives RecursionError, and the paper has asked for exactly that fault to be identified.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
count = 5

def reset():
    count = 0

reset()
print(count)
```

- a. 0
- b. 5
- c. None
- d. an error

**X** 0

Wrong. Assigning inside a function makes a new local name.

**✓** 5

Correct. The count inside reset() is local to it, so the outer count is untouched.

**X** None

Wrong. The outer count still holds 5.

**X** an error

Wrong. It runs without error.

A name assigned inside a function is local and disappears when the function ends: what happens in the function stays in the function.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
with open("t.txt", "w") as f:
    f.write("x\n")
with open("t.txt") as f:
    print(repr(f.readline()))
```

- a. x
- b. 'x\n'
- c. 'x'
- d. ['x\n']

**X** x

Wrong. repr shows the quotes and the newline marker.

**✓** 'x\n'

Correct. readline() gives the next line as a string, still carrying its newline, which repr shows as \n.

**X** 'x'

Wrong. The newline is kept.

**X** ['x\n']

Wrong. readline() gives a string, not a list.

**|** readline() returns one line including its newline; readlines() returns a list of such strings.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
with open("t.txt", "w") as f:  
    n = f.write("hello\n")  
print(n)
```

- a. hello
- b. 6
- c. 5
- d. None

**X** hello

Wrong. It returns a number, not the text.

**✓** 6

Correct. write() returns the number of characters written: five letters and the newline.

**X** 5

Wrong. The newline counts as a character.

**X** None

Wrong. write() returns a count.

**|** f.write(s) writes the text and returns how many characters it wrote.

**Asked twice.** This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.2 and Lesson 3.5.

**What happens when this runs?**

```
try:
    d = {"a": 1}
    print(d["b"])
except ValueError:
    print("bad value")
```

- a. It prints None
- b. It prints bad value
- c. It prints nothing and carries on
- d. A KeyError stops the program, because except ValueError does not catch it

✗ It prints None

Wrong. `d["b"]` raises; it does not give None.

✗ It prints bad value

Wrong. The lookup raises `KeyError`, not `ValueError`.

✗ It prints nothing and carries on

Wrong. An exception that is not caught stops the program.

✓ A `KeyError` stops the program, because `except ValueError` does not catch it

Correct. `except` only catches the exception it names; this one is a `KeyError`.

■ Name the exception you expect. An exception you did not name passes straight through.

**Asked twice.** This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4.

**What does this print?**

```
def withdraw(balance, amount):  
    if amount > balance:  
        raise ValueError("insufficient funds")  
    return balance - amount  
  
try:  
    print(withdraw(50, 80))  
except ValueError as e:  
    print("Error:", e)
```

- a. an unhandled ValueError
- b. Error: insufficient funds
- c. -30
- d. insufficient funds

**X** an unhandled ValueError

Wrong. The ValueError is caught.

**✓** Error: insufficient funds

Correct. raise signals the error, and the except catches it and prints its message.

**X** -30

Wrong. The raise stops the function before it subtracts.

**X** insufficient funds

Wrong. The print adds "Error:" in front.

**|** raise signals an error yourself; except ValueError as e gives you its message in e.

## 5 Databases and GUI Development

5.1

EASY

Manual O.5 Q1

What is the correct order for working with a database in Python?

- a. connect, cursor, execute, commit, close
- b. cursor, connect, commit, execute, close
- c. connect, execute, cursor, close, commit
- d. open, read, write, close

✓ connect, cursor, execute, commit, close

Correct. Connect to the database, get a cursor from the connection, execute SQL through the cursor, commit so the change is saved, then close the connection.

✗ cursor, connect, commit, execute, close

Wrong. A cursor is obtained FROM a connection, so the connection must exist first.

✗ connect, execute, cursor, close, commit

Wrong. You cannot execute before you have a cursor, and committing after closing is too late.

✗ open, read, write, close

Wrong. That is the file-handling pattern, not the database one.

The answer is connect, cursor, execute, commit, close. connect, cursor, execute, commit, close.

Forget the commit and the rows vanish when the program ends, which is the classic lost mark.

A program inserts three rows and then closes the connection without calling commit.  
What is in the table?

- a. All three rows
- b. No rows
- c. The first row only
- d. An error is raised

**X** All three rows

Wrong. Uncommitted work is not saved. It lived only in the transaction.

**✓** No rows

Correct. Without a commit the transaction is rolled back when the connection closes, so the table is unchanged.

**X** The first row only

Wrong. There is nothing special about the first row; all three are in the same uncommitted transaction.

**X** An error is raised

Wrong. Nothing raises. The program appears to work, which is what makes this bug expensive.

The answer is No rows. Commit is what makes a change permanent. No commit, no data, and no error message to tell you.

Which SQL statement retrieves only the students whose score is above 50?

- a. SELECT \* FROM students IF score > 50
- b. GET \* FROM students WHERE score > 50
- c. SELECT \* FROM students WHERE score > 50
- d. SELECT students WHERE score > 50

✗ SELECT \* FROM students IF score > 50

Wrong. SQL filters with WHERE. IF is not part of a SELECT statement.

✗ GET \* FROM students WHERE score > 50

Wrong. There is no GET in SQL. Retrieval is SELECT.

✓ SELECT \* FROM students WHERE score > 50

Correct. SELECT names the columns, FROM names the table, WHERE filters the rows.

✗ SELECT students WHERE score > 50

Wrong. The table must be named after FROM, and the columns before it.

The answer is SELECT \* FROM students WHERE score > 50. SELECT columns FROM table WHERE condition. That one line is most of what this course asks of SQL.

After executing a **SELECT** that matches five rows, what does `fetchone()` return?

- a. All five rows as a list
- b. The first row, as a tuple
- c. The number 5
- d. None

✗ All five rows as a list

Wrong. That is `fetchall()`. `fetchone` gets one.

✓ The first row, as a tuple

Correct. `fetchone()` hands back the next row as a tuple, and returns `None` once the rows run out.

✗ The number 5

Wrong. Counting rows is done in SQL, with `COUNT(*)`.

✗ None

Wrong. `None` comes back only when there are no rows left, and here there are five.

The answer is The first row, as a tuple. `fetchone` next row or `None`, `fetchall` the rest as a list of tuples. A row is a tuple, so index it by position.

Which SQL function returns the number of rows matching a query?

- a. SUM()
- b. TOTAL()
- c. COUNT()
- d. LEN()

✗ SUM()

Wrong. SUM() adds up the values in a column. It answers how much, not how many.

✗ TOTAL()

Wrong. There is no standard TOTAL() in this course.

✓ COUNT()

Correct. COUNT(\*) counts rows; COUNT(column) counts the rows where that column is not null.

✗ LEN()

Wrong. LEN() is not SQL. Python has len(); SQL has COUNT().

The answer is COUNT(). COUNT how many, SUM the total, AVG the mean, MAX and MIN the extremes. The paper has asked for all five.

What does `UPDATE students SET score = 0` do, with no `WHERE` clause?

- a. Nothing: a `WHERE` clause is required
- b. It sets score to 0 for the first row only
- c. It deletes the score column
- d. It sets score to 0 for EVERY row in the table

✗ Nothing: a `WHERE` clause is required

Wrong. `WHERE` is optional, and that is precisely the danger.

✗ It sets score to 0 for the first row only

Wrong. SQL is set-based: it acts on every row that matches, and with no `WHERE` every row matches.

✗ It deletes the score column

Wrong. Removing a column is `ALTER TABLE`. `UPDATE` changes values.

✓ It sets score to 0 for EVERY row in the table

Correct. With no `WHERE`, the statement applies to the whole table. The same is true of `DELETE FROM students`, which empties it.

The answer is It sets score to 0 for EVERY row in the table. No `WHERE` means every row. Write the `WHERE` first and the `SET` second if that helps you never forget it.

In tkinter, what does the `mainloop()` call do?

- a. It starts the event loop, which waits for and dispatches user actions
- b. It draws the window once and returns
- c. It creates the main window object
- d. It closes the application

✓ It starts the event loop, which waits for and dispatches user actions

Correct. `mainloop()` keeps the program running, watching for clicks and keystrokes and calling the handlers bound to them, until the window is closed.

✗ It draws the window once and returns

Wrong. Drawing once and returning would make the window appear and vanish immediately, which is exactly what happens if you forget `mainloop`.

✗ It creates the main window object

Wrong. Creating the window is `Tk()`. `mainloop` is what makes it live.

✗ It closes the application

Wrong. It is what keeps the application OPEN.

The answer is It starts the event loop, which waits for and dispatches user actions. `Tk()` makes the window, widgets are created and placed, `mainloop()` runs the event loop. Without the last one nothing stays on screen.

Which tkinter widget is used to collect a single line of typed text?

- a. Label
- b. Button
- c. Entry
- d. Canvas

✗ Label

Wrong. A Label DISPLAYS text. The user cannot type into it.

✗ Button

Wrong. A Button is clicked; it does not accept typing.

✓ Entry

Correct. Entry is the one-line text box, and its `get()` method reads what was typed.

✗ Canvas

Wrong. A Canvas is a drawing surface for shapes and images.

The answer is Entry. Label shows, Entry takes one line, Text takes many, Button acts. Read what was typed with `entry.get()`.

In `Button(root, text="Go", command=run)`, when is `run` called?

- a. Immediately, as the button is created
- b. When `mainloop()` starts
- c. Never, unless `bind()` is also used
- d. When the button is clicked

✗ Immediately, as the button is created

Wrong, and this is the trap. It WOULD be immediate if you wrote `command=run()`, with brackets: that calls the function and hands its return value to `command`.

✗ When `mainloop()` starts

Wrong. `mainloop` starts the waiting; it does not fire the handlers by itself.

✗ Never, unless `bind()` is also used

Wrong. `command=` is a complete way to attach a handler to a button. `bind()` is for other events.

✓ When the button is clicked

Correct. `command` is given the function ITSELF, without brackets, and `tkinter` calls it later, each time the button is pressed.

The answer is When the button is clicked. Pass the function, not a call to it. `command=run` attaches it; `command=run()` runs it once immediately and attaches its result.

Which SQL column type suits a score that can have a decimal part?

- a. REAL
- b. TEXT
- c. BLOB
- d. INTEGER

✓ REAL

Correct. REAL is SQL's floating point type.

✗ TEXT

Wrong. TEXT holds strings.

✗ BLOB

Wrong. BLOB holds raw binary data.

✗ INTEGER

Wrong. INTEGER holds whole numbers only.

| The four SQL types: INTEGER, TEXT, REAL, BLOB.

**Why write CREATE TABLE IF NOT EXISTS rather than plain CREATE TABLE?**

- a. So the columns get types automatically
- b. So commit() is not needed
- c. So running the program a second time does not fail because the table is already there
- d. So the table is emptied each time

**X** So the columns get types automatically

Wrong. You still write the column types.

**X** So commit() is not needed

Wrong. commit() is still needed to keep inserted rows.

**✓** So running the program a second time does not fail because the table is already there

Correct. Without it, the second run fails, and the paper asks for tables created 'if they do not already exist'.

**X** So the table is emptied each time

Wrong. It does not empty anything.

**|** IF NOT EXISTS makes the create safe to run again.

You are laying out a form of labels beside entry boxes. Which geometry manager suits it?

- a. `place()`, because it uses exact pixels
- b. `grid()`, because a form is rows and columns
- c. `pack()`, because it stacks widgets
- d. `mainloop()`

✗ `place()`, because it uses exact pixels

Wrong. `place` works, but fixing every pixel is fragile.

✓ `grid()`, because a form is rows and columns

Correct. `grid` places widgets by row and column numbers, which is what a form is.

✗ `pack()`, because it stacks widgets

Wrong. `pack` stacks widgets in the order added; it does not line up columns.

✗ `mainloop()`

Wrong. `mainloop` starts the event loop; it does not arrange anything.

| `pack` stacks, `grid` uses rows and columns, `place` uses exact x and y.

Which geometry manager positions a widget at exact x and y coordinates?

- a. grid()
- b. pack()
- c. place()
- d. config()

✗ grid()

Wrong. grid uses rows and columns.

✗ pack()

Wrong. pack stacks widgets.

✓ place()

Correct. place uses exact pixel coordinates.

✗ config()

Wrong. config changes a widget's settings.

! The three geometry managers: pack, grid and place.

Which widget gives the user a MULTI-line area to type in?

- a. Button
- b. Label
- c. Text
- d. Entry

✗ Button

Wrong. A Button is clicked, not typed in.

✗ Label

Wrong. A Label displays text the user cannot edit.

✓ Text

Correct. Text is a multi line typing area.

✗ Entry

Wrong. Entry takes a single line.

Entry for one line, Text for several, Label to show, Button to click.

### What is a Frame used for?

- a. Starting the event loop
- b. Reading typed text
- c. Grouping other widgets together as a container
- d. Showing a picture only

✗ Starting the event loop

Wrong. That is `mainloop()`.

✗ Reading typed text

Wrong. That is `entry.get()`.

✓ Grouping other widgets together as a container

Correct. A Frame is a container that groups other widgets.

✗ Showing a picture only

Wrong. Not its purpose in the course.

Frame groups; Label shows; Entry and Text take typing; Button clicks.

Which two of these are tkinter geometry managers?

- a. pack()
- b. grid()
- c. mainloop()
- d. Entry()

✓ pack()

Correct. pack is a geometry manager.

✓ grid()

Correct. grid is a geometry manager.

✗ mainloop()

Wrong. mainloop starts the event loop.

✗ Entry()

Wrong. Entry is a widget.

| pack, grid and place arrange widgets; mainloop keeps the window alive.

**5.17****EASY**

Lesson 3.6

How does a program read what the user typed into an Entry called entry?

- a. entry.mainloop()
- b. entry.get()
- c. entry.read()
- d. input(entry)

✗ entry.mainloop()

Wrong. mainloop belongs to the window.

✓ entry.get()

Correct. get() returns the typed text.

✗ entry.read()

Wrong. read() is for files.

✗ input(entry)

Wrong. input() reads the text console, not a widget.

| entry.get() reads what the user typed.

**5.18****EASY**

Lesson 3.6

True or false: without window.mainloop(), the window appears and vanishes at once.

Answer: **true**

| No loop, no window: mainloop starts the event loop that waits for clicks and typing.

Why is `cur.execute("INSERT INTO t VALUES (?, ?)", (a, b))` preferred to building the SQL string with the values in it?

- a. It runs faster on small tables
- b. It is the only form sqlite3 accepts
- c. It commits automatically
- d. It keeps the values separate from the SQL, so a value cannot be read as SQL

✗ It runs faster on small tables

Wrong. Speed is not the reason, though prepared statements can help on repeated inserts.

✗ It is the only form sqlite3 accepts

Wrong. String building is accepted. It is accepted and unsafe, which is the point.

✗ It commits automatically

Wrong. You still have to commit yourself.

✓ It keeps the values separate from the SQL, so a value cannot be read as SQL

Correct. The placeholder keeps data as data. Pasting a value into the text of a statement lets a value containing a quote change the statement, which is SQL injection.

The answer is It keeps the values separate from the SQL, so a value cannot be read as SQL. The question mark placeholder separates data from code. It is the standard answer to "how do you prevent SQL injection".

What does `cur.executemany()` do that `execute()` does not?

- a. It runs the same statement once for each item in a sequence of parameter tuples
- b. It runs several DIFFERENT statements in one call
- c. It commits after every row
- d. It returns all the rows of a table

✓ It runs the same statement once for each item in a sequence of parameter tuples

Correct. You give it one statement and a list of parameter tuples, and it runs the statement once per tuple, which is how a batch insert is done.

✗ It runs several DIFFERENT statements in one call

Wrong. The statement is the same every time. Only the parameters change.

✗ It commits after every row

Wrong. It does not commit at all. You still call commit yourself.

✗ It returns all the rows of a table

Wrong. Retrieving rows is `fetchall()`, after a `SELECT`.

The answer is It runs the same statement once for each item in a sequence of parameter tuples. One statement, many parameter sets. It is the tidy way to insert a list of records and this manual teaches it because the paper reached for it.

Which of these is NOT a tkinter geometry manager?

- a. align
- b. pack
- c. grid
- d. place

✓ align

Correct. There is no align manager. The three are pack, grid and place, and you should not mix pack and grid in the same container.

✗ pack

Wrong choice, because pack IS one: it stacks widgets in the space available.

✗ grid

Wrong choice, because grid IS one: it places widgets by row and column.

✗ place

Wrong choice, because place IS one: it positions by exact coordinates.

The answer is align. pack, grid, place. Three of them, and mixing pack with grid in one container makes the window hang while they argue.

**Asked twice.** This same question also appears as: Lesson 3.5 and Lesson 3.5 and Lesson 3.5.

**The same table. What does this print?**

```
import sqlite3
conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE t (name TEXT, score REAL)")
cur.executemany("INSERT INTO t VALUES (?, ?)", [("Ada", 70), ("Ben", 55), ("Cy", 82)])
conn.commit()

cur.execute("UPDATE t SET score = 50")
cur.execute("SELECT score FROM t")
print(cur.fetchall())
```

- a. [(50.0,), (50.0,), (50.0,)]
- b. [(70.0,), (55.0,), (82.0,)]
- c. [(50.0,), (55.0,), (82.0,)]
- d. [(50.0,)]

✓ [(50.0,), (50.0,), (50.0,)]

Correct. With no WHERE, UPDATE changes every row in the table.

✗ [(70.0,), (55.0,), (82.0,)]

Wrong. The UPDATE ran and changed the scores.

✗ [(50.0,), (55.0,), (82.0,)]

Wrong. Nothing limits the UPDATE to the first row.

✗ [(50.0,)]

Wrong. All three rows are still there, all changed.

**WHERE or everywhere:** UPDATE and DELETE without WHERE apply to every row.

**Asked twice.** This same question also appears as: Lesson 3.5 and Lesson 3.5 and Lesson 3.5.

**The same table. What does this print?**

```
import sqlite3
conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE t (name TEXT, score REAL)")
cur.executemany("INSERT INTO t VALUES (?, ?)", [("Ada", 70), ("Ben", 55), ("Cy", 82)])
conn.commit()

cur.execute("DELETE FROM t WHERE score < ?", (60,))
cur.execute("SELECT COUNT(*) FROM t")
print(cur.fetchone()[0])
```

a. 1

b. 3

c. 0

d. 2

**X** 1

Wrong. Ada and Cy both stay.

**X** 3

Wrong. The DELETE removed Ben.

**X** 0

Wrong. WHERE limits the DELETE to matching rows.

**✓** 2

Correct. Only Ben's row has a score below 60, so one row goes and two remain.

**|** DELETE FROM t WHERE ... removes only the matching rows.

**Asked twice.** This same question also appears as: Lesson 3.5 and Lesson 3.5 and Lesson 3.5.

**The same table. What does this print?**

```
import sqlite3
conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE t (name TEXT, score REAL)")
cur.executemany("INSERT INTO t VALUES (?, ?)", [("Ada", 70), ("Ben", 55), ("Cy", 82)])
conn.commit()

cur.execute("SELECT name FROM t WHERE score > 100")
print(cur.fetchone())
```

- a. ()
- b. []
- c. None
- d. an error

**X** ()

Wrong. It returns None, not an empty tuple.

**X** []

Wrong. That is what fetchall() returns for no rows.

**✓** None

Correct. fetchone() returns None when there is no row to give.

**X** an error

Wrong. An empty result is not an error.

**|** fetchone() gives the next row as a tuple, or None when the rows run out.

What does the course warn happens if you mix `pack()` and `grid()` in the same container?

- a. The window closes immediately
- b. The widgets are drawn twice
- c. Nothing; they combine neatly
- d. The program can freeze

✗ The window closes immediately

Wrong. That is what happens without `mainloop()`.

✗ The widgets are drawn twice

Wrong. Not what the course says.

✗ Nothing; they combine neatly

Wrong. That is exactly what the rule forbids.

✓ The program can freeze

Correct. The rule is never to mix `pack` and `grid` in one container.

Two rules: no `mainloop`, no window; and never mix `pack` with `grid` inside the same container.

The table `t` holds Ada 70, Ben 55 and Cy 82. What does this print?

```
import sqlite3
conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE t (name TEXT, score REAL)")
cur.executemany("INSERT INTO t VALUES (?, ?)", [("Ada", 70), ("Ben", 55), ("Cy", 82)])
conn.commit()

cur.execute("SELECT name FROM t WHERE score > ? ORDER BY score DESC", (60,))
print(cur.fetchall())
```

- a. [('Ada',), ('Cy',)]
- b. [('Cy',), ('Ada',)]
- c. [('Cy',), ('Ada',), ('Ben',)]
- d. ['Cy', 'Ada']

✗ [('Ada',), ('Cy',)]

Wrong. ORDER BY score DESC puts 82 before 70.

✓ [('Cy',), ('Ada',)]

Correct. Only scores above 60 match, ordered highest first, and each row comes back as a one-item tuple.

✗ [('Cy',), ('Ada',), ('Ben',)]

Wrong. Ben's 55 fails the WHERE.

✗ ['Cy', 'Ada']

Wrong. fetchall() returns a list of TUPLES.

fetchall() gives every matching row as a list of tuples, even when each row has one column.

**Asked twice.** This same question also appears as: Lesson 3.5 and Lesson 3.5 and Lesson 3.5.

**The same table. What does this print?**

```
import sqlite3
conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE t (name TEXT, score REAL)")
cur.executemany("INSERT INTO t VALUES (?, ?)", [("Ada", 70), ("Ben", 55), ("Cy", 82)])
conn.commit()

cur.execute("SELECT COUNT(*), MAX(score), AVG(score) FROM t")
print(cur.fetchone())
```

- a. 3 82.0 69.0
- b. (3, 82, 69)
- c. [(3, 82.0, 69.0)]
- d. (3, 82.0, 69.0)

✗ 3 82.0 69.0

Wrong. The row is printed as a tuple.

✗ (3, 82, 69)

Wrong. score is REAL, so MAX and AVG are floats.

✗ [(3, 82.0, 69.0)]

Wrong. fetchone() gives one row, not a list.

✓ (3, 82.0, 69.0)

Correct. The three functions come back as ONE row, a tuple, and the REAL column gives floats.

█ COUNT(\*), MAX() and AVG() in one query return a single row, collected with fetchone().

**Asked twice.** This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.2 and Lesson 3.4.

### What happens when this runs?

```
import sqlite3
conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE t (name TEXT, score REAL)")
cur.executemany("INSERT INTO t VALUES (?, ?)", [("Ada", 70), ("Ben", 55), ("Cy", 82)])
conn.commit()

cur.execute("SELECT * FROM t WHERE name = ?", ("Ada"))
print(cur.fetchone())
```

- a. It prints every row
- b. It prints Ada's row
- c. An error, because ("Ada") is only a string; the placeholder needs the tuple ("Ada",)
- d. It prints None

✗ It prints every row

Wrong. The query fails.

✗ It prints Ada's row

Wrong. The parameters are wrong, so the query never runs.

✓ An error, because ("Ada") is only a string; the placeholder needs the tuple ("Ada",)

Correct. Without the comma the brackets are just grouping, so SQLite receives three characters instead of one value.

✗ It prints None

Wrong. The error happens before any fetch.

| ("SS2") is a string; ("SS2",) is a tuple, and the placeholder needs the tuple.